Sound Monomorphisation and Monotonicity-Based Guard Erasure for Hammers for Dependent Type Theory

AI draft of July 7, 2026

Abstract

Hammers connect proof assistants to automated theorem provers (ATPs) by translating goals and selected premises into first-order logic. For dependent type theory the standard translation, implemented in CoqHammer, is an untyped encoding in which typing information is retained through *guards*: atoms $\mathsf{T}(x, \tau)$ expressing that the individual x has type τ. Guards make the encoding sound but verbose, and quantification over types makes the resulting problems hard for ATPs. Two optimisations promise substantial improvements: *heuristic monomorphisation*, which instantiates type quantifiers with ground types occurring in the problem, and *guard erasure*, which deletes guards that provably do not affect provability. Both have long been used for simple type theory in Sledgehammer-style tools, where their correctness rests on the model theory of many-sorted first-order logic. No comparable analysis exists for dependent type theory; the standard soundness proof for the CoqHammer translation is proof-theoretic and, as its author observed, incompatible with the model-theoretic methods underlying monotonicity inference.

This paper closes that gap. We define a calculus CIC^- — a predicative fragment of the Calculus of Inductive Constructions with an impredicative universe of propositions and parametric inductive types — together with a set-theoretic semantics, and give a self-contained, model-theoretic soundness proof for a guarded translation of CIC^- into classical untyped first-order logic: the guard predicate is interpreted as set membership, and every first-order consequence of the translated problem is true in the induced standard model. On this basis we prove three groups of results. First, heuristic monomorphisation — formulated as premise instantiation at the level of CIC^-, followed by normalisation and retranslation — preserves soundness unconditionally, and, when generic premises are kept, provability; we also show that no complete recursive monomorphisation procedure exists. Second, *signature specialisation*, the per-occurrence replacement of guard, application and truth atoms by fresh specialised symbols (without bridging axioms), preserves soundness, and is provability-preserving on fully monomorphised problems. Third, we formulate *monotonicity-based guard erasure* for the resulting monomorphic problems and prove it sound: guards of a type τ may be erased whenever τ is witnessed inhabited and either no variable of type τ occurs naked in a positive equation, or the problem forces τ to be infinite — a condition that for inductive types is established by the injectivity and discrimination axioms the translation itself emits. We prove that both side conditions are necessary, and that the classical counterexample to guard erasure in the CoqHammer literature is precisely a violation of monotonicity. All proofs are given in full.

Contents

1 Introduction

Hammers — tools that discharge proof obligations of an interactive proof assistant by calling automated theorem provers (ATPs) — have become standard equipment for proof assistants based on simple type theory, and increasingly for those based on dependent type theory: CoqHammer for Coq/Rocq [CK18], and more recently Lean-auto and the Lean hammer effort [QCBA25, ZCA+26]. A hammer has three phases: premise selection, translation of the goal and premises into the ATPs' logic, and reconstruction of the found proof inside the assistant. This paper is about the middle phase for dependent type theory, and specifically about two optimisations that are folklore for simple type theory but have never been given a rigorous treatment for the Calculus of Inductive Constructions (CIC):

- *Heuristic monomorphisation*: replacing premises that quantify over types ("$\forall A{:}\mathsf{Type}.\ \forall x{:}A.\ \forall l{:}\mathsf{list}\,A.\ \ldots$") by finitely many instances at the ground types occurring in the problem ($A \coloneqq \mathsf{nat}$, …). For Sledgehammer-style hammers this is known to outperform not only the polymorphic encodings but even native polymorphic prover support [BBPS16, DVBW22]; for CIC it has remained a to-do item.

- *Guard erasure*: the CoqHammer translation is an untyped encoding that retains typing information as *guards* — atoms $\mathsf{T}(x, \tau)$ read "x has type τ" — attached to every quantifier. Guards multiply literals and clutter every clause. For many-sorted first-order logic, the monotonicity analysis of Claessen, Lilliestr\"om and Smallbone [CLS11] and Blanchette, Böhme, Popescu and Smallbone [BBPS16] identifies which sorts' guards can be erased soundly. Whether and how that analysis transfers to dependent type theory was posed as an open question in the soundness study of the CoqHammer translation [Cza18, §6], whose own proof-theoretic method is incompatible with the model constructions monotonicity requires. Meanwhile the implementation omits certain guards on empirical grounds, with a known concrete counterexample delimiting the practice [CK18, §5.6].

Both optimisations are *dangerous*: instantiation interacts with the type-directed translation, and guard erasure strengthens premises — erase the guard of a quantifier over a one-element type and everything collapses to a point (Proposition 7.19). The purpose of this paper is to say precisely when they are safe, and to prove it.

Approach. We work over CIC$^-$ (Section 2), a stratified fragment of CIC with an impredicative universe of propositions, two predicative universes, primitive connectives, and parametric inductive data types — rich enough to express polymorphic premises, their instances, and all the guard phenomena at issue, while excluding proof-dependent data (we prove proofs are hermetically confined in the fragment, Lemma 2.7). The key methodological decision is semantic: we fix a set-theoretic standard model of the environment (Section 3, following [Wer97, Acz99, LW11]) and prove a *bidirectional truth lemma* (Theorem 4.10) for a guarded translation in which the guard predicate is interpreted as *set membership* (Section 4). Every transformation of the paper is then measured against one invariant: the translated axioms must remain true in the induced first-order structure, and truth of the translated goal must continue to imply validity of the original goal (Definition 4.12). This model-theoretic setup is exactly what the erasure analysis needs and what the existing proof-theoretic soundness framework [Cza18] cannot provide; it also matches the actual use of *classical* ATPs, for which minimal-logic proof reconstruction does not apply. The guarantee it delivers is semantic — optimised ATP successes imply the goal is true — and combines with the kernel-checked reconstruction phase, which makes the end-to-end system safe unconditionally (Remark 8.5).

Contributions.

1. A self-contained, model-theoretic soundness theorem for a guarded CIC-to-FOL translation in the style of CoqHammer (Theorems 4.10 and 4.11), including a polarity analysis showing which of the implemented translation's guard forms are validated by the semantics and which are not (Remark 4.13).

2. A formulation of heuristic monomorphisation as premise instantiation at the CIC$^-$ level — instantiate, normalise, retranslate — with: soundness independent of the instance-selection heuristic (Theorem 5.6); conservativity of the keep-generic policy (Proposition 5.7); incompleteness of the drop-generic policy (Proposition 5.8); coverage of higher-order instances (Remark 5.10); and a proof that no computable complete instance-selection strategy exists for CIC$^-$ (Theorem 5.14), adapting the Bobot–Paskevich combinatory-logic reduction [BP11] to a semantic consequence relation, with a full decidability analysis of the instance fragment.

3. *Signature specialisation* (Section 6): a uniform per-occurrence renaming — type-argument fusion, guard specialisation $\mathsf{T}(x, \tau) \mapsto \mathsf{P}_\tau(x)$, arity flattening, predicate

flattening — proved sound by one interpretation principle (Theorem 6.2) and proved *faithful* (provability-preserving in both directions, Theorem 6.4) on the first-order core, *without the bridging axioms* the current implementation emits.

4. *Monotonicity-based guard erasure* (Section 7): a formulation of guard erasure for the fully monomorphised, fully specialised problems; the reduction of its soundness to *inflatability* (Theorem 7.5) with unconditional proof-preservation (Proposition 7.3); a sorted-core extraction and relativisation theorem (Theorem 7.9); full adaptations of the cloning and merging model constructions (Lemma 7.13 and Theorem 7.17); an *internal* infinity criterion for inductive types driven by the translation's own injectivity and discrimination axioms (Lemma 7.15); and proofs that both side conditions — witnessed inhabitation and monotonicity — are necessary (Propositions 7.18 and 7.19), the latter subsuming the known counterexample of [CK18, §5.6] (Remark 7.20).

5. The composed pipeline with its end-to-end soundness theorem and an account of exactly where completeness is traded away (Section 8).

All proofs are given in full, except for standard metatheory of Pure Type Systems and inductive types (Proposition 2.6) and the existence of set-theoretic models (Proposition 3.4), which are quoted with references.

Design constraint. Throughout, transformations are *shallow*: per-occurrence manipulations of formulas, without mediating axiomatic bridges between old and new symbols. This constraint, adopted from the CoqHammer development guidelines, is not cosmetic: bridging axioms reintroduce the generic apparatus into every problem and defeat the purpose of specialisation. The faithfulness and erasure theorems show that on the fragment that matters — fully monomorphised, essentially first-order problems — nothing is lost by shallowness.

Organisation. Section 2 defines CIC^- and proves the two fragment lemmas. Section 3 fixes the semantics. Section 4 defines the translation and proves the truth lemma and base soundness. Sections 5 to 7 develop the three optimisations, and Section 8 composes them. Section 9 discusses related work.

2 The calculus CIC^-

We work with a calculus CIC^-, a predicative fragment of the Calculus of Inductive Constructions with an impredicative universe of propositions, primitive logical connectives, and parametric inductive data types. The design follows the intermediate representation CIC_0 of [CK18] in having an explicit syntactic distinction between propositions and data, and primitive connectives; it deviates from full CIC in ways that we record explicitly in Remark 2.10 and discuss throughout. The two restrictions that carry mathematical weight are the absence of proof-dependent data (Lemma 2.7) and the restriction of inductive types to parametric data types.

2.1 Syntax

Definition 2.1 (Sorts and terms)**.** The set of *sorts* is $\mathcal{S} = \{\mathsf{Prop}, \mathsf{Type}_1, \mathsf{Type}_2\}$. The set of *terms* is generated by the grammar

$$\begin{aligned} t, u, A, B, \varphi, \psi ::= {} & x \mid c \mid s \mid \Pi x{:}A.\, B \mid \lambda x{:}A.\, t \mid t\, u \mid \mathsf{case}_{I,\vec{r},Q}(t;\, \vec{u}) \\ & \mid \top \mid \bot \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \leftrightarrow \psi \mid t \doteq_A u \mid \exists x{:}A.\, \varphi \end{aligned}$$

where x ranges over variables, c over constants and s over sorts. We write $A \to B$ for $\Pi x{:}A.\, B$ when $x \notin \mathrm{FV}(B)$, and $\neg\varphi$ for $\varphi \to \bot$. Universal quantification is not a separate former: $\forall x{:}A.\, \varphi$ *is* the dependent product $\Pi x{:}A.\, \varphi$. Terms are identified up to α-conversion; $\mathrm{FV}(t)$ denotes the set of free variables; $t[u/x]$ denotes capture-avoiding substitution, and $t[\vec{u}/\vec{x}]$ its simultaneous variant.

Definition 2.2 (Environments)**.** A *global environment* E is a finite sequence of declarations of the following forms:

(i) a *definition* $c \coloneqq t : T$;

(ii) a *parameter declaration* $c : T$ (a constant without a body; premises available to the hammer are of this form once their proof terms are discarded);

(iii) an *inductive declaration*

$$\mathsf{Ind}\big(I;\ A_1{:}\mathsf{Type}_1, \ldots, A_p{:}\mathsf{Type}_1;\ c_1 : \Pi\vec{A}{:}\mathsf{Type}_1.\, \sigma_{1,1} \to \cdots \to \sigma_{1,n_1} \to I\,\vec{A},\ \ldots,\ c_k : \cdots\big)$$

introducing a *data inductive type* I with $p \geq 0$ parameters and $k \geq 0$ constructors $c_1, \ldots, c_k$. Each constructor argument type $\sigma_{j,l}$ is required to be either (a) a term with $A_1{:}\mathsf{Type}_1, \ldots, A_p{:}\mathsf{Type}_1 \vdash \sigma_{j,l} : \mathsf{Type}_1$ in which I does not occur, or (b) exactly the term $I\,\vec{A}$ (a *recursive argument*).

A *context* Γ is a finite sequence $x_1{:}A_1, \ldots, x_n{:}A_n$ of variable declarations. The environment is a global parameter of all judgements below; we keep it implicit and write $\Gamma \vdash t : T$ for the typing judgement.

The inductive scheme of Definition 2.2(iii) covers the parametric data types relevant to monomorphisation — nat, bool, unit, $\mathsf{list}\,A$, $\mathsf{prod}\,A\,B$, $\mathsf{option}\,A$, binary trees, and so on — while excluding indexed families and nested or higher-order recursive occurrences. Constructor arguments do not depend on one another. Inductively defined *propositions* (Forall, le, …) are not part of the calculus: an inductive predicate enters a problem as a parameter constant $c : \vec{\sigma} \to \mathsf{Prop}$ together with premises axiomatising it — for instance its introduction rules and its inversion principle, all of which are provable propositions of the ambient proof assistant. This is not a loss of generality for our purposes: the soundness of any axiom that is the translation of a *provable* proposition follows from the premise-truth part of Theorem 4.11 below, so inductive predicates require no dedicated treatment. Only *data* inductive types need built-in support, because the translation emits genuinely new axioms for them (injectivity, discrimination, inversion), and precisely those axioms are the subject of the guard-erasure analysis of Section 7.

2.2 Reduction and conversion

Definition 2.3 (Reduction)**.** The relation $\to_{\beta\iota\delta}$ is the contextual closure of:

- (β) $(\lambda x{:}A.\, t)\, u \to_{\beta\iota\delta} t[u/x]$;
- (ι) $\mathsf{case}_{I,\vec{\tau},Q}(c_j\,\vec{\tau}\,\vec{v};\, \vec{u}) \to_{\beta\iota\delta} u_j\,\vec{v}$, where c_j is the j-th constructor of I;
- (δ) $c \to_{\beta\iota\delta} t$, for every definition $c \coloneqq t : T$ in the environment.

$\to^*_{\beta\iota\delta}$ is its reflexive–transitive closure and $\simeq$ (*conversion*) the generated equivalence. We write $t \Downarrow u$ if $t \to^*_{\beta\iota\delta} u$ and u is in normal form.

2.3 Typing

Definition 2.4 (Typing)**.** The judgement $\Gamma \vdash t : T$ is generated by the following rules. The PTS core uses the axioms $\mathcal{A} = \{\mathsf{Prop} : \mathsf{Type}_1,\ \mathsf{Type}_1 : \mathsf{Type}_2\}$ and the rule set

$$\mathcal{R} \;=\; \{\,(s, \mathsf{Prop}, \mathsf{Prop}) \mid s \in \mathcal{S}\,\} \;\cup\; \{\,(\mathsf{Type}_i, \mathsf{Type}_j, \mathsf{Type}_{\max(i,j)}) \mid i, j \in \{1,2\}\,\}.$$

$$(\textsc{ax})\ \frac{(s_1 : s_2) \in \mathcal{A}}{\Gamma \vdash s_1 : s_2} \qquad (\textsc{var})\ \frac{(x{:}A) \in \Gamma \quad \Gamma \vdash A : s}{\Gamma \vdash x : A} \qquad (\textsc{const})\ \frac{(c : T) \text{ or } (c \coloneqq t : T) \text{ in } E}{\Gamma \vdash c : T}$$

$$(\textsc{prod})\ \frac{\Gamma \vdash A : s_1 \quad \Gamma, x{:}A \vdash B : s_2 \quad (s_1, s_2, s_3) \in \mathcal{R}}{\Gamma \vdash \Pi x{:}A.\, B : s_3}$$

$$(\textsc{lam})\ \frac{\Gamma, x{:}A \vdash t : B \quad \Gamma \vdash \Pi x{:}A.\, B : s}{\Gamma \vdash \lambda x{:}A.\, t : \Pi x{:}A.\, B}$$

$$(\textsc{app})\ \frac{\Gamma \vdash t : \Pi x{:}A.\, B \quad \Gamma \vdash u : A}{\Gamma \vdash t\, u : B[u/x]} \qquad (\textsc{conv})\ \frac{\Gamma \vdash t : A \quad \Gamma \vdash B : s \quad A \simeq B}{\Gamma \vdash t : B}$$

For an inductive declaration as in Definition 2.2(iii), with $\vec{\tau}$ a vector of p terms:

$$(\textsc{ind})\ \frac{}{\Gamma \vdash I : \underbrace{\mathsf{Type}_1 \to \cdots \to \mathsf{Type}_1}_{p} \to \mathsf{Type}_1}$$

$$(\textsc{constr})\ \frac{}{\Gamma \vdash c_j : \Pi \vec{A}{:}\mathsf{Type}_1.\, \sigma_{j,1} \to \cdots \to \sigma_{j,n_j} \to I\,\vec{A}}$$

$$(\textsc{case})\ \frac{\begin{array}{c}\Gamma \vdash t : I\,\vec{\tau} \qquad \Gamma \vdash Q : I\,\vec{\tau} \to s \qquad s \in \{\mathsf{Prop}, \mathsf{Type}_1\} \\ \Gamma \vdash u_j : \sigma_{j,1}[\vec{\tau}/\vec{A}] \to \cdots \to \sigma_{j,n_j}[\vec{\tau}/\vec{A}] \to Q\,(c_j\,\vec{\tau}\,\vec{y}) \ \text{ for } j = 1, \ldots, k\end{array}}{\Gamma \vdash \mathsf{case}_{I,\vec{\tau},Q}(t;\, \vec{u}) : Q\,t}$$

(the branch typing is understood as: u_j has the displayed closed product type, in which $\vec{y}$ are the bound argument variables). Finally, the logical formers:

$$\frac{}{\Gamma \vdash \top : \mathsf{Prop}} \qquad \frac{}{\Gamma \vdash \bot : \mathsf{Prop}} \qquad \frac{\Gamma \vdash \varphi : \mathsf{Prop} \quad \Gamma \vdash \psi : \mathsf{Prop} \quad \circ \in \{\wedge, \vee, \leftrightarrow\}}{\Gamma \vdash \varphi \circ \psi : \mathsf{Prop}}$$

$$\frac{\Gamma \vdash A : \mathsf{Type}_1 \quad \Gamma \vdash t : A \quad \Gamma \vdash u : A}{\Gamma \vdash t \doteq_A u : \mathsf{Prop}} \qquad \frac{\Gamma \vdash A : \mathsf{Type}_1 \quad \Gamma, x{:}A \vdash \varphi : \mathsf{Prop}}{\Gamma \vdash \exists x{:}A.\, \varphi : \mathsf{Prop}}$$

Well-formedness of contexts and environments is defined as usual: each declared type must be typable by a sort (for definitions, the body must have the declared type), and the constructor argument condition of Definition 2.2(iii) must hold. We consider only well-formed environments and contexts from now on.

Note which products exist. Since $\mathcal{R}$ contains no rule of the form $(\mathsf{Prop}, \mathsf{Type}_i, -)$, there are *no functions from proofs to data or to types*: a product $\Pi x{:}\varphi.\, B$ with $\varphi : \mathsf{Prop}$ is well-formed only when $B : \mathsf{Prop}$. Quantification over propositions ($\Pi A{:}\mathsf{Prop}.\, \varphi$, sort of the domain Type_1) and over data types ($\Pi A{:}\mathsf{Type}_1.\, \varphi$, sort of the domain Type_2) into Prop are both available, giving the impredicativity of Prop familiar from CIC. Polymorphic data such as $\Pi A{:}\mathsf{Type}_1.\, A \to \mathsf{list}\, A$ lives in Type_2; type constructors such as $\mathsf{list} : \mathsf{Type}_1 \to \mathsf{Type}_1$ have kinds in Type_2. There is no cumulativity: Prop, Type_1 and Type_2 are unrelated by subtyping.

Definition 2.5 (Classification)**.** Let $\Gamma \vdash t : T$. We call t: a *proposition* if $T \simeq \mathsf{Prop}$[1]; a *proof* if $\Gamma \vdash T : \mathsf{Prop}$; a *data type* if $T \simeq \mathsf{Type}_1$; a *data object* if $\Gamma \vdash T : \mathsf{Type}_1$; and *kind-level* otherwise (this covers Type_1 itself, kinds such as $\mathsf{Type}_1 \to \mathsf{Type}_1$, type constructors,

[1] Here and below, statements such as "$T \simeq \mathsf{Prop}$" are justified by uniqueness of types (Proposition 2.6): the alternative types of t agree up to conversion, and distinct sorts are distinct normal forms, so the classification does not depend on the choice of T.

polymorphic functions, and Type_2-sorted products generally). A variable x with $(x{:}A) \in \Gamma$ is a *proof variable* if A is a proposition, a *data variable* if A is a data type, and a *type variable* if $A = \mathsf{Type}_1$.

2.4 Metatheory

The following standard properties are inherited from the general theory of Pure Type Systems and of CIC; we state them for reference and use them freely. CIC$^-$ is a functional (singly sorted) PTS extended with parametric inductive types satisfying the usual strict-positivity condition, so the results below are instances of the literature [Bar92, Geu93, Wer94, PM93].

Proposition 2.6 (Standard metatheory)**.**

(1) (Confluence) $\to_{\beta\iota\delta}$ *is confluent on well-typed terms.*

(2) (Substitution) *If* $\Gamma, x{:}A, \Delta \vdash t : T$ *and* $\Gamma \vdash u : A$*, then* $\Gamma, \Delta[u/x] \vdash t[u/x] : T[u/x]$.

(3) (Subject reduction) *If* $\Gamma \vdash t : T$ *and* $t \to_{\beta\iota\delta} t'$ *then* $\Gamma \vdash t' : T$.

(4) (Uniqueness of types) *If* $\Gamma \vdash t : T_1$ *and* $\Gamma \vdash t : T_2$ *then* $T_1 \simeq T_2$.

(5) (Strong normalisation) *Every well-typed term is strongly* $\beta\iota\delta$*-normalising.*

(6) (Sort classification) *If* $\Gamma \vdash t : T$*, then either* $T \simeq \mathsf{Type}_2$*, or there is a unique sort* $s \in \mathcal{S}$ *with* $\Gamma \vdash T : s$*. Consequently the five classes of Definition 2.5 are mutually exclusive and exhaustive on well-typed terms.*

The next two lemmas are specific to CIC$^-$ and carry real weight in this paper, so we prove them in full. The first says that proofs are hermetically confined: a term that is not itself a proof contains no proofs anywhere inside it. This is what makes a *total*, proof-free translation possible on the fragment (Section 4), and it plays the role that proof irrelevance plays in the piPTS soundness proof of [Cza18]: the known counterexample to soundness of shallow embeddings without proof irrelevance ([Cza18, Example 52]: a predicate $q : p \to \mathsf{Prop}$ applied to two different proof variables) is not expressible in CIC$^-$, because the type $p \to \mathsf{Prop}$ with $p : \mathsf{Prop}$ requires the excluded rule $(\mathsf{Prop}, \mathsf{Type}_2, \cdot)$.

Lemma 2.7 (Proof confinement)**.** *Let* $\Gamma \vdash t : T$ *and suppose* t *is not a proof. Then no subterm occurrence of* t *(including type annotations of binders) is a proof relative to its local context, and in particular no free or bound variable of* t *is a proof variable.*

Proof. By induction on the structure of t, using the generation (inversion) properties of the typing rules. We check that every immediate subterm of t, in its local context, is again a non-proof; the statement then follows by the induction hypothesis applied to the immediate subterms.

Case t *a variable, constant or sort:* immediate (no proper subterms).

Case $t = u\,v$*:* by generation, $\Gamma \vdash u : \Pi x{:}A.\, B$ with $\Gamma \vdash v : A$ and $T \simeq B[v/x]$. First, u is not a proof: the type $\Pi x{:}A.\, B$ of u has some sort s_3 with $(s_1, s_2, s_3) \in \mathcal{R}$, and if $s_3 = \mathsf{Prop}$ then, by the shape of $\mathcal{R}$, $s_2 = \mathsf{Prop}$; then $B : \mathsf{Prop}$, hence $B[v/x] : \mathsf{Prop}$ by Proposition 2.6(2), so t would be a proof, contradiction. So $s_3 \neq \mathsf{Prop}$ and u is not a proof. Second, v is not a proof: if A were a proposition, i.e. $A : \mathsf{Prop}$, then the product rule used for $\Pi x{:}A.\, B$ has $s_1 = \mathsf{Prop}$, and the only rules in $\mathcal{R}$ with first component Prop are $(\mathsf{Prop}, \mathsf{Prop}, \mathsf{Prop})$; thus $B : \mathsf{Prop}$ and again t is a proof, contradiction.

Case $t = \lambda x{:}A.\, u$*:* by generation $T \simeq \Pi x{:}A.\, B$ with $\Gamma, x{:}A \vdash u : B$, and $\Pi x{:}A.\, B : s_3$ with $s_3 \neq \mathsf{Prop}$ since t is not a proof. As in the previous case, $s_3 \neq \mathsf{Prop}$ forces $s_1 \neq \mathsf{Prop}$ (all rules with $s_1 = \mathsf{Prop}$ have $s_3 = \mathsf{Prop}$), so A is not a proposition and x is not a proof variable;

moreover B's sort s_2 satisfies $(s_1, s_2, s_3) \in \mathcal{R}$ with $s_3 \neq \mathsf{Prop}$, hence $s_2 \neq \mathsf{Prop}$, so u is not a proof in $\Gamma, x{:}A$. The annotation A has type $s_1 \neq \mathsf{Prop}$, so A is not a proof either.

Case $t = \Pi x{:}A.\, B$*:* t has a sort as its type, so t is never a proof, and its immediate subterms A and B have sorts as types, so neither is a proof.

Case $t = \mathsf{case}_{I,\vec{\tau},Q}(u; \vec{u})$*:* the scrutinee satisfies $\Gamma \vdash u : I\,\vec{\tau} : \mathsf{Type}_1$, so u is a data object, not a proof. The parameters $\vec{\tau}$ are data types (Type_1-typed), not proofs. The motive Q has type $I\,\vec{\tau} \to s$, a kind or a Type_2-sorted product, so Q is not a proof. The branches u_j have Type_2- or Type_1-sorted product types when $s = \mathsf{Type}_1$; when $s = \mathsf{Prop}$ they have types of sort Prop — but then $Q\,u : \mathsf{Prop}$ and t itself is a proof, contradicting the hypothesis. So when t is not a proof, $s = \mathsf{Type}_1$ and no immediate subterm is a proof.

Case t *a connective, equation or existential:* $t : \mathsf{Prop}$, so t is a proposition, not a proof. Immediate subterms: subformulas are propositions; the sides of $t_1 \doteq_A t_2$ are data objects and A a data type; the domain of $\exists x{:}A.\, \varphi$ is a data type and the body a proposition. None are proofs.

For the "in particular" part: a free proof variable would itself be a proof subterm occurrence; a bound proof variable requires a binder $\lambda x{:}\varphi$ or $\Pi x{:}\varphi$ or $\exists x{:}\varphi.$ with $\varphi : \mathsf{Prop}$ inside t. The λ-case was excluded above whenever the λ is not a proof; a subterm λ that *is* a proof cannot occur, since we showed no subterm of t is a proof. A product $\Pi x{:}\varphi.\, \psi$ with a proposition domain forces $\psi : \mathsf{Prop}$; such a subterm is an implication $\varphi \to \psi$: its bound variable cannot occur in ψ, because $x \in \mathrm{FV}(\psi)$ with x a proof variable would make some subterm of ψ contain the free proof variable x, and ψ is a proposition, hence not a proof, so by the induction hypothesis ψ has no free proof variables. The existential former binds only data variables by its typing rule. □

Corollary 2.8. *If* $\Gamma \vdash \Pi x{:}\varphi.\, \psi : \mathsf{Prop}$ *with* $\Gamma \vdash \varphi : \mathsf{Prop}$*, then* $x \notin \mathrm{FV}(\psi)$*. Products over propositions are plain implications.*

The second lemma states that instantiating a type variable by a closed data type leaves the classification of every subterm unchanged. It is the reason why the type-directed translation of Section 4 interacts predictably with the monomorphisation of Section 5.

Lemma 2.9 (Sort invariance under type instantiation)**.** *Let* $\Gamma, A{:}\mathsf{Type}_1, \Delta \vdash t : T$ *and let* τ *be a term with* $\Gamma \vdash \tau : \mathsf{Type}_1$*. Then* $\Gamma, \Delta[\tau/A] \vdash t[\tau/A] : T[\tau/A]$*, and* $t[\tau/A]$ *belongs to the same class of Definition 2.5 (proposition, proof, data type, data object, kind-level) as* t*.*

Proof. The typing statement is Proposition 2.6(2). For the classification, first note that sorts are closed terms, so $s[\tau/A] = s$ for every sort s.

If t is kind-level with $T \simeq \mathsf{Type}_2$, then $T[\tau/A] \simeq \mathsf{Type}_2[\tau/A] = \mathsf{Type}_2$ (conversion is closed under substitution), so $t[\tau/A]$ is kind-level. Otherwise, by Proposition 2.6(6) there is a unique sort s with $\Gamma, A{:}\mathsf{Type}_1, \Delta \vdash T : s$. By Proposition 2.6(2) applied to this judgement, $\Gamma, \Delta[\tau/A] \vdash T[\tau/A] : s[\tau/A] = s$. By uniqueness of types (Proposition 2.6(4),(6)), s is *the* sort of the type of $t[\tau/A]$. Hence: t a proof (sort of T is Prop) iff $t[\tau/A]$ a proof; t a data object iff $t[\tau/A]$ a data object; and similarly for kind-level terms whose type has sort Type_2.

For the remaining two classes, which are defined by the type itself rather than by its sort: if $T \simeq \mathsf{Prop}$ then $T[\tau/A] \simeq \mathsf{Prop}$, so propositions map to propositions; if $T \simeq \mathsf{Type}_1$ then $T[\tau/A] \simeq \mathsf{Type}_1$, so data types map to data types. Conversely, no new memberships in these classes can appear: the classes are mutually exclusive and exhaustive (Proposition 2.6(6)), and we have just shown each class to be preserved, so the induced map on classes is the identity. □

Remark 2.10 (Relation to full CIC). Compared to the calculus underlying Coq/Rocq, CIC^- omits: cumulativity and the full universe hierarchy (we keep two predicative levels, which suffice to type polymorphic premises and their instances; universe polymorphism and the

constraints machinery are orthogonal to our concerns — note that the implemented translation *drops* universe constraints altogether [CK18, §3], which is one of its known sources of theoretical unsoundness, whereas CIC$^-$ is honestly stratified); proof-dependent data, i.e. the rules ($\mathsf{Prop}, \mathsf{Type}_i, \mathsf{Type}_i$), hence strong sums such as `sig` and proof-carrying records; indexed inductive families and nested/mutual/higher-order recursion; general `fix` (recursive functions can be accommodated as parameter constants whose guarded defining equations enter as premises — the translation of [CK18] in any case converts `fix` into exactly such equations, and our soundness framework only requires the equations to be *valid*, cf. Definition 3.2); and η-conversion. The implemented CoqHammer translation handles the omitted features by opaque naming ("lifting"); nothing in the monomorphisation and guard-erasure analysis below depends on them.

3 Set-theoretic semantics

Our soundness framework is model-theoretic. This is a deliberate choice, and a departure from the existing rigorous treatment of the CoqHammer translation: the soundness proof of [Cza18] for the core translation is proof-theoretic — it reconstructs a type-theoretic proof term from a first-order proof in *minimal intuitionistic* first-order logic by induction on an η-long normal derivation. That technique does not extend to the questions of this paper, for two reasons. First, the ATPs actually invoked by a hammer are *classical*: a classical refutation is not a minimal-logic derivation, so the reconstruction argument does not even apply to the proofs the ATPs find. Second, and decisively, the optimisations we study — monotonicity-based guard erasure above all — are justified by *model constructions* (domain inflation, sort merging), and, as [Cza18, §6] itself observes, such model-theoretic methods are incompatible with the proof-theoretic soundness argument. We therefore set up a set-theoretic semantics for CIC$^-$ once and for all, and measure every transformation against it. The price is that our soundness guarantee is semantic ("the goal is true in the standard model") rather than proof-theoretic ("a proof term exists"); in the hammer setting this is the appropriate guarantee, since the final arbiter is in any case the proof-assistant kernel: the ATP output is used only to select premises, and the goal is re-proved from them by certified search [CK18]. What the semantic guarantee rules out is the pipeline systematically wasting effort on, and reporting success for, goals that are false; equivalently, it establishes that each transformation adds no inconsistency to the generated problems.

Throughout, the metatheory is ZFC with two strongly inaccessible cardinals; the axiom of choice is used and we flag the places where it matters.

Definition 3.1 (Standard model)**.** Fix Grothendieck universes $\mathcal{U}_1 \in \mathcal{U}_2$ (i.e. $\mathcal{U}_1 = V_{\kappa_1}$, $\mathcal{U}_2 = V_{\kappa_2}$ for inaccessibles $\kappa_1 < \kappa_2$), and let $\Omega = \{\emptyset, \{\emptyset\}\}$ where we write $\{\emptyset\}$ for $\{\emptyset\}$; so $\Omega = \{\emptyset, \{\emptyset\}\} \in \mathcal{U}_1$. Call a function ρ from variables to sets a Γ-*assignment* if $\rho(x) \in [\![A]\!]_\rho$ for every $(x{:}A) \in \Gamma$ (well-defined by recursion on the context). A *standard model* $\mathcal{M}$ of an environment E is a function assigning to every judgement $\Gamma \vdash t : T$ and every Γ-assignment ρ a set $[\![t]\!]_\rho$ (independent of the derivation and of T; we omit ρ for closed terms), such that:

(M1) *(Typing soundness)* $[\![t]\!]_\rho \in [\![T]\!]_\rho$ whenever $\Gamma \vdash t : T$. Moreover $[\![t]\!]_\rho = \emptyset$ whenever t is a proof.

(M2) *(Sorts)* $[\![\mathsf{Prop}]\!] = \Omega$, $[\![\mathsf{Type}_1]\!] = \mathcal{U}_1$, $[\![\mathsf{Type}_2]\!] = \mathcal{U}_2$.

(M3) *(Variables, constants)* $[\![x]\!]_\rho = \rho(x)$; for a definition $c \coloneqq t : T$, $[\![c]\!] = [\![t]\!]$.

(M4) *(Products)* If $\Gamma, x{:}A \vdash B : \mathsf{Prop}$ then

$$[\![\Pi x{:}A.\, B]\!]_\rho \;=\; \{\, z \in \{\emptyset\} \mid \forall a \in [\![A]\!]_\rho.\ [\![B]\!]_{\rho[x \mapsto a]} = \{\emptyset\} \,\}.$$

Otherwise (the product has sort Type_1 or Type_2), $[\![\Pi x{:}A.\,B]\!]_\rho$ is the set-theoretic dependent product: the set of all functions (graphs) f with domain $[\![A]\!]_\rho$ such that $f(a) \in [\![B]\!]_{\rho[x\mapsto a]}$ for all $a \in [\![A]\!]_\rho$.

(M5) *(Abstraction, application)* If $\lambda x{:}A.\,t$ is not a proof, then $[\![\lambda x{:}A.\,t]\!]_\rho$ is the graph $\{\,(a, [\![t]\!]_{\rho[x\mapsto a]}) \mid a \in [\![A]\!]_\rho\,\}$; if $t\,u$ is not a proof, then $[\![t\,u]\!]_\rho = [\![t]\!]_\rho([\![u]\!]_\rho)$ (function application of the graph).

(M6) *(Conversion)* If $\Gamma \vdash t : T$, $\Gamma \vdash t' : T$ and $t \simeq t'$, then $[\![t]\!]_\rho = [\![t']\!]_\rho$.

(M7) *(Substitutivity)* If $\Gamma, x{:}A, \Delta \vdash t : T$ and $\Gamma \vdash u : A$, then $[\![t[u/x]]\!]_\rho = [\![t]\!]_{\rho[x\mapsto [\![u]\!]_\rho]}$ for every assignment ρ conforming to $\Gamma, \Delta[u/x]$.

(M8) *(Connectives)* $[\![\top]\!] = \{\emptyset\}$, $[\![\bot]\!] = \emptyset$, and for Γ-propositions:

$$
\begin{aligned}
[\![\varphi \wedge \psi]\!]_\rho &= \{\, z \in \{\emptyset\} \mid [\![\varphi]\!]_\rho = \{\emptyset\} \text{ and } [\![\psi]\!]_\rho = \{\emptyset\}\,\},\\
[\![\varphi \vee \psi]\!]_\rho &= \{\, z \in \{\emptyset\} \mid [\![\varphi]\!]_\rho = \{\emptyset\} \text{ or } [\![\psi]\!]_\rho = \{\emptyset\}\,\},\\
[\![\varphi \leftrightarrow \psi]\!]_\rho &= \{\, z \in \{\emptyset\} \mid [\![\varphi]\!]_\rho = [\![\psi]\!]_\rho\,\},\\
[\![t \doteq_A u]\!]_\rho &= \{\, z \in \{\emptyset\} \mid [\![t]\!]_\rho = [\![u]\!]_\rho\,\},\\
[\![\exists x{:}A.\,\varphi]\!]_\rho &= \{\, z \in \{\emptyset\} \mid \exists a \in [\![A]\!]_\rho.\ [\![\varphi]\!]_{\rho[x\mapsto a]} = \{\emptyset\}\,\}.
\end{aligned}
$$

(M9) *(Inductive types)* For each inductive declaration as in Definition 2.2(iii), $[\![I]\!]$ is a function $\mathcal{U}_1^p \to \mathcal{U}_1$, and for each j the constructor denotation $[\![c_j]\!]$ is a (curried) function taking $\vec{a} \in \mathcal{U}_1^p$ and then argument tuples, such that for all $\vec{a} \in \mathcal{U}_1^p$:

(a) *(Closure and leastness)* $[\![I]\!](\vec{a})$ is the least set $X \in \mathcal{U}_1$ such that for every j and every tuple $\vec{d}$ with $d_l \in [\![\sigma_{j,l}]\!]_{[\vec{A}\mapsto\vec{a}]}$ for non-recursive positions and $d_l \in X$ for recursive positions, the value $[\![c_j]\!](\vec{a})(\vec{d})$ belongs to X.

(b) *(Injectivity)* each $[\![c_j]\!](\vec{a})$ is injective (jointly in all its value arguments).

(c) *(Disjointness)* for $i \neq j$ the images of $[\![c_i]\!](\vec{a})$ and $[\![c_j]\!](\vec{a})$ on well-typed argument tuples are disjoint.

(d) *(Exhaustiveness)* $[\![I]\!](\vec{a})$ equals the union over j of the images of $[\![c_j]\!](\vec{a})$ on well-typed argument tuples.

(M10) *(Case)* If $\Gamma \vdash \mathsf{case}_{I,\vec{\tau},Q}(t;\,\vec{u}) : Q\,t$ is not a proof and $[\![t]\!]_\rho = [\![c_j]\!]([\![\vec{\tau}]\!]_\rho)(\vec{d})$ (such j and $\vec{d}$ exist and are unique by (M9)(b–d)), then $[\![\mathsf{case}_{I,\vec{\tau},Q}(t;\,\vec{u})]\!]_\rho = [\![u_j]\!]_\rho(\vec{d})$ (iterated application); if the case term is a proof its denotation is $\emptyset$ by (M1).

(M11) *(Parameters)* For every parameter declaration $c : T$ in E: $[\![c]\!] \in [\![T]\!]$.

Definition 3.2 (Admissible environment; validity)**.** An environment E is *admissible* if a standard model of it exists. Fix an admissible E and a standard model $\mathcal{M}$. A closed proposition φ is *valid* (in $\mathcal{M}$) if $[\![\varphi]\!] = \{\emptyset\}$.

Note that admissibility already builds in the truth of all premises: if $c : \varphi$ is a parameter declaration with φ a closed proposition, then (M11) gives $[\![c]\!] \in [\![\varphi]\!] \in \Omega$, hence $[\![\varphi]\!] = \{\emptyset\}$, i.e. φ is valid. The same holds for anything *derivable*:

Proposition 3.3 (Provability implies validity)**.** *If $\vdash t : \varphi$ for a closed proposition φ, then φ is valid.*

Proof. By (M1), $[\![t]\!] \in [\![\varphi]\!]$, so $[\![\varphi]\!] \neq \emptyset$; since $[\![\varphi]\!] \in [\![\mathsf{Prop}]\!] = \Omega$ by (M1) applied to $\varphi : \mathsf{Prop}$, we get $[\![\varphi]\!] = \{\emptyset\}$. □

Proposition 3.4 (Existence)**.** *Every environment arising from a consistent development in the standard set-theoretic interpretation of CIC is admissible. More precisely, the proof-irrelevant set-theoretic model constructions of Werner [Wer97], Aczel [Acz99] and Lee–Werner [LW11] interpret a calculus containing* CIC^- *in ZFC with inaccessibles and satisfy (M1)–(M11); in particular, environments consisting of well-typed definitions, data inductive declarations, and parameter declarations whose types are interpretable (e.g. premises that are provable, or axioms true in the intended interpretation, such as functional extensionality, proof irrelevance, excluded middle or classical choice) are admissible.*

Proof notes. We do not reproduce the model construction; we indicate why each clause of Definition 3.1 holds in it. Products into Prop are interpreted by the trace encoding exactly as in (M4) (this is Werner's proof-irrelevant interpretation; [Wer97, LW11]); products into type universes are set-theoretic dependent products, with $\mathcal{U}_1$ closed under them by inaccessibility of κ_1 (for a product over a proposition domain into a type universe there is nothing to check — such products are not well-formed in CIC^-). (M6) and (M7) are the standard soundness lemmas of the interpretation. Connectives and the primitive equality (M8) match the interpretation of the corresponding inductive definitions of CIC under proof irrelevance: e.g. Coq's `eq` is interpreted as a family that is $\{\emptyset\}$ on the diagonal and $\emptyset$ off it. Inductive data types are interpreted as least fixed points of their constructor rules with constructors modelled by injectively tagged tuples, giving (M9)(a–d); ι-reduction soundness gives (M10). (M11) is the definition of interpretability of the parameters. □

Remark 3.5. Everything that follows is relative to one fixed standard model $\mathcal{M}$ of the ambient environment. Which standard model is chosen is irrelevant to the theorems; what matters is that all premises given to the hammer are valid in it, which admissibility guarantees. Note also a convenient consequence of (M9): concrete data types have their expected denotations. For instance the empty inductive type (no constructors) denotes $\emptyset$; a one-constructor, zero-argument type (unit) denotes a singleton; $[\![\mathsf{nat}]\!]$ is Dedekind-infinite, since $[\![\mathsf{S}]\!]$ is injective by (M9)(b) and omits $[\![\mathsf{O}]\!]$ from its image by (M9)(c); and $[\![\mathsf{list}]\!](a)$ is infinite whenever $a \neq \emptyset$, finite (a singleton) when $a = \emptyset$. These facts drive the counterexamples and the infinity criterion of Section 7.

4 The guarded translation and its model-theoretic soundness

This section defines the base translation of CIC^- problems into classical untyped first-order logic and proves it sound with respect to the standard model. The translation follows the shape of the CoqHammer translation [CK18, §5.2] and of the core embedding of [Cza18], with one systematic design decision, discussed in Remark 4.13: guards attached to *hypothesis* (negative) positions are always *membership* guards $\mathsf{T}(x, \mathsf{C}(A))$, while typing information emitted in *conclusion* (positive) positions may be unfolded pointwise. The payoff of this discipline is a bidirectional truth lemma (Theorem 4.10) in a model where T is literally set membership.

4.1 The target logic

Definition 4.1 (First-order signature and semantics)**.** The target is classical first-order logic with equality $\approx$ over the signature Σ_{FOL} containing: a binary function symbol $@$ (*application*, written infix and left-associatively, $t \mathbin{@} u$, often just $t\,u$); a binary predicate T (*the guard predicate*); a unary predicate Pr (*truth of an encoded proposition*); an individual constant $\hat{c}$ for every constant c of the environment (including inductive type names and constructors), constants $\ulcorner\mathsf{Prop}\urcorner$ and $\ulcorner\mathsf{Type}_1\urcorner$, and the *lifted constants* introduced in Definition 4.3 below. We use standard Tarskian semantics; $\mathfrak{M}, \nu \models \varphi$ denotes truth of φ in the structure $\mathfrak{M}$ under

the valuation ν, and $\Phi \vdash_{\mathrm{c}} \varphi$ denotes classical derivability. By Gödel completeness, $\Phi \vdash_{\mathrm{c}} \varphi$ iff every model of Φ satisfies φ. A *problem* is a pair (Φ, φ) of a set of sentences (axioms) and a sentence (the conjecture); an ATP establishes $\Phi \vdash_{\mathrm{c}} \varphi$ by refuting $\Phi \cup \{\neg\varphi\}$.

Convention 4.2. Every CIC$^-$ variable is also a first-order variable. Proof variables never occur in translated material (Lemma 2.7), so no "proof-erasure" constant is needed — a simplification over [CK18, Cza18], where a constant prf/ε stands for erased proofs.

4.2 Lifted constants

The term encoding must map binders (products, abstractions, case expressions) and propositions-in-term-position to first-order *terms*. As in [CK18, Cza18] this is done by *lifting*: the offending subterm is replaced by a new constant applied to the encodings of its free variables, and axioms describing the constant are emitted. To make the translation a well-defined function (and to model the hash-consing performed by the implementation) we index lifted constants by what they lift.

Definition 4.3 (Lifted constants; extended environment)**.** Let $\Gamma \vdash t : T$ where t is not a proof, and let $\vec{y}{:}\vec{\rho} = \mathrm{FC}(\Gamma; t)$ be the subcontext of Γ declaring the free variables of t (in context order; by Lemma 2.7 none of them are proof variables). The *lifted constant* $\ulcorner t \urcorner_{\vec{y}:\vec{\rho}}$ is a first-order constant indexed by the α-equivalence class of the pair $(\vec{y}{:}\vec{\rho},\, t)$; distinct classes yield distinct constants. The *extended environment* E^+ adds, for every such pair used by the translation, the CIC$^-$ definition

$$\ell_{(\vec{y}:\vec{\rho};\, t)} \;\coloneqq\; \lambda \vec{y}{:}\vec{\rho}.\, t \;:\; \Pi \vec{y}{:}\vec{\rho}.\, T.$$

E^+ is admissible whenever E is: the new constants are definitions, interpreted by (M3) as $[\![\lambda \vec{y}{:}\vec{\rho}.\, t]\!]$. We write $\mathcal{M}$ also for the extension of the standard model to E^+, and we identify the first-order constant $\ulcorner t \urcorner_{\vec{y}:\vec{\rho}}$ with $\hat{\ell}_{(\vec{y}:\vec{\rho};\, t)}$.

4.3 The translation functions

Definition 4.4 (Guards)**.** For a Γ-term A that is not a proposition (a data type, Type_1, or kind-level), and a first-order term u, the *membership guard* is

$$\mathsf{G}_\Gamma(u, A) \;\coloneqq\; \mathsf{T}(u, \mathsf{C}_\Gamma(A)).$$

For a context $\Delta = x_1{:}A_1, \ldots, x_n{:}A_n$ extending Γ and a formula φ, the *guarded closure* is

$$\langle \forall \Delta \rangle_\Gamma\, \varphi \;\coloneqq\; Q_1\,(Q_2\,(\cdots Q_n\,\varphi)), \qquad Q_i \;=\; \begin{cases} \forall x_i.\ \mathsf{G}_{\Gamma_i}(x_i, A_i) \to - & \text{if } A_i \text{ not a proposition,} \\ \mathsf{F}_{\Gamma_i}(A_i) \to - & \text{if } A_i \text{ a proposition,} \end{cases}$$

where $\Gamma_i = \Gamma, x_1{:}A_1, \ldots, x_{i-1}{:}A_{i-1}$. The *flattened typing formula* of a first-order term u at a Γ-type T (not a proposition) is defined by recursion on T:

$$\mathrm{Flat}_\Gamma(u, \Pi x{:}A.\, B) \;=\; \forall x.\ \mathsf{G}_\Gamma(x, A) \to \mathrm{Flat}_{\Gamma, x:A}(u \mathbin{@} x,\ B), \qquad \mathrm{Flat}_\Gamma(u, T) \;=\; \mathsf{T}(u, \mathsf{C}_\Gamma(T))$$

for T not a product. (In a non-propositional product $\Pi x{:}A.\, B$ of CIC$^-$ the domain A is never a proposition, by the shape of $\mathcal{R}$; so the first clause needs no proposition case. Flat coincides with the guard function G of [CK18, §5.2].)

Definition 4.5 (Translation)**.** For Γ-propositions φ the formula translation $\mathsf{F}_\Gamma(\varphi)$, and for Γ-terms t that are neither proofs nor sorts other than $\mathsf{Prop}, \mathsf{Type}_1$ the term translation $\mathsf{C}_\Gamma(t)$, are defined by mutual recursion. Each clause of $\mathsf{C}()$ that introduces a lifted constant also *emits* axioms into the axiom store Δ; emission is idempotent (axioms are indexed by the same α-classes as the constants).

Formula translation $\mathsf{F}_\Gamma(\varphi)$:

(F1) $\mathsf{F}_\Gamma(\Pi x{:}A.\,\varphi) = \mathsf{F}_\Gamma(A) \to \mathsf{F}_\Gamma(\varphi)$ if A is a proposition (then $x \notin \mathrm{FV}(\varphi)$ by Corollary 2.8);

(F2) $\mathsf{F}_\Gamma(\Pi x{:}A.\,\varphi) = \forall x.\ \mathsf{G}_\Gamma(x, A) \to \mathsf{F}_{\Gamma,x:A}(\varphi)$ if A is not a proposition;

(F3) $\mathsf{F}_\Gamma(\top) = \top$, $\mathsf{F}_\Gamma(\bot) = \bot$, $\mathsf{F}_\Gamma(\varphi \circ \psi) = \mathsf{F}_\Gamma(\varphi) \circ \mathsf{F}_\Gamma(\psi)$ for $\circ \in \{\wedge, \vee, \leftrightarrow\}$;

(F4) $\mathsf{F}_\Gamma(t \doteq_A u) = \big(\mathsf{C}_\Gamma(t) \approx \mathsf{C}_\Gamma(u)\big)$;

(F5) $\mathsf{F}_\Gamma(\exists x{:}A.\,\varphi) = \exists x.\ \mathsf{G}_\Gamma(x, A) \wedge \mathsf{F}_{\Gamma,x:A}(\varphi)$;

(F6) otherwise (φ atomic: a variable, constant, application or case expression of type Prop): $\mathsf{F}_\Gamma(\varphi) = \mathsf{Pr}(\mathsf{C}_\Gamma(\varphi))$.

Term translation $\mathsf{C}_\Gamma(t)$:

(C1) $\mathsf{C}_\Gamma(x) = x$; $\mathsf{C}_\Gamma(c) = \hat{c}$; $\mathsf{C}_\Gamma(\mathsf{Prop}) = \ulcorner\mathsf{Prop}\urcorner$; $\mathsf{C}_\Gamma(\mathsf{Type}_1) = \ulcorner\mathsf{Type}_1\urcorner$;

(C2) $\mathsf{C}_\Gamma(t\,u) = \mathsf{C}_\Gamma(t) \mathbin{@} \mathsf{C}_\Gamma(u)$ (by Lemma 2.7 neither t nor u is a proof);

(C3) *(proposition lifting)* if t is a proposition that is not a variable, constant or application whose translation is wanted as a term (i.e. t occurs in term position), then $\mathsf{C}_\Gamma(t) = \ulcorner t \urcorner_{\vec{y}:\vec{\rho}}\,\vec{y}$ with $\vec{y}{:}\vec{\rho} = \mathrm{FC}(\Gamma; t)$, emitting

$$\langle \forall\, \vec{y}{:}\vec{\rho} \rangle \Big(\mathsf{Pr}\big(\ulcorner t \urcorner\, \vec{y}\big) \leftrightarrow \mathsf{F}_\Gamma(t) \Big); \tag{Δ-prop}$$

for t a variable, constant or application, clauses (C1)–(C2) apply unchanged also when t is a proposition;

(C4) *(product lifting)* if $t = \Pi x{:}A.\,B$ is not a proposition (so it is a data type or kind-level), then $\mathsf{C}_\Gamma(t) = \ulcorner t \urcorner_{\vec{y}:\vec{\rho}}\,\vec{y}$, emitting the *application-typing axiom*

$$\langle \forall\, \vec{y}{:}\vec{\rho} \rangle \Big(\forall z.\ \mathsf{T}\big(z, \ulcorner t \urcorner\, \vec{y}\big) \to \forall x.\ \mathsf{G}_\Gamma(x, A) \to \mathsf{T}\big(z \mathbin{@} x,\ \mathsf{C}_{\Gamma,x:A}(B)\big) \Big); \tag{Δ-app}$$

(C5) *(abstraction lifting)* if $t = \lambda\vec{x}{:}\vec{A}.\,b$ with maximal λ-prefix (so b is not an abstraction), t not a proof, then $\mathsf{C}_\Gamma(t) = \ulcorner t \urcorner_{\vec{y}:\vec{\rho}}\,\vec{y}$, emitting

$$\langle \forall\, \vec{y}{:}\vec{\rho},\ \vec{x}{:}\vec{A} \rangle \Big(\ulcorner t \urcorner\, \vec{y}\, \vec{x} \ \approx_{\Gamma'}\ b \Big) \tag{Δ-λ}$$

where $\Gamma' = \Gamma, \vec{x}{:}\vec{A}$ and $\approx_{\Gamma'}$ is the *levelled equality*: for first-order term u and Γ'-term b,

$$u \approx_{\Gamma'} b \ :=\ \begin{cases} \mathsf{Pr}(u) \leftrightarrow \mathsf{F}_{\Gamma'}(b) & \text{if } b \text{ is a proposition,} \\ \forall z.\ \mathsf{T}(z, u) \leftrightarrow \mathsf{T}(z, \mathsf{C}_{\Gamma'}(b)) & \text{if } b \text{ is a data type or kind-level,} \\ u \approx \mathsf{C}_{\Gamma'}(b) & \text{if } b \text{ is a data object;} \end{cases}$$

additionally the *typing atoms and flattened typing axiom* for $\ulcorner t \urcorner$ are emitted:

$$\langle \forall\, \vec{y}{:}\vec{\rho} \rangle\, \mathsf{T}\big(\ulcorner t \urcorner\, \vec{y},\ \mathsf{C}_\Gamma(T)\big), \qquad \langle \forall\, \vec{y}{:}\vec{\rho} \rangle\, \mathrm{Flat}_\Gamma\big(\ulcorner t \urcorner\, \vec{y},\ T\big) \tag{Δ-ty}$$

where T is the type of t (a non-propositional product, since t is not a proof);

(C6) *(case lifting)* if $t = \mathsf{case}_{I,\vec{\tau},Q}(u;\,\vec{u})$ is not a proof and not a proposition-in-term-position handled by (C3), then $\mathsf{C}_\Gamma(t) = \ulcorner t \urcorner_{\vec{y}:\vec{\rho}}\,\vec{y}$, emitting the axioms ($\Delta$-ty) for its type $Q\,u$ — when that type is not a product the flattened axiom degenerates to the typing atom — and the *case equation*

$$\langle \forall\, \vec{y}{:}\vec{\rho} \rangle \bigvee_{j=1}^{k} \exists \vec{x}_j.\ \Big(\textstyle\bigwedge_l \mathsf{G}(x_{j,l}, \sigma_{j,l}[\vec{\tau}/\vec{A}]) \ \wedge\ \mathsf{C}_\Gamma(u) \approx \hat{c}_j\, \mathsf{C}_\Gamma(\vec{\tau})\, \vec{x}_j \ \wedge\ \big(\ulcorner t \urcorner\, \vec{y} \approx_{\Gamma_j} u_j\, \vec{x}_j \big) \Big) \tag{Δ-case}$$

with $\Gamma_j = \Gamma, \vec{x}_j : \vec{\sigma}_j[\vec{\tau}/\vec{A}]$. Note that the guards on $\vec{y}$ — in particular on the free variables of the scrutinee u — are part of the axiom; these are the guards that [CK18, §5.6] identifies as never omittable, and Proposition 7.19 below reproves that observation semantically.

Definition 4.6 (Translation of declarations and problems)**.** Fix an admissible environment. For each kind of declaration the following axioms are generated (all in the empty context).

(D1) *Parameter* $c : T$. If T is a proposition: the *premise formula* $\mathsf{F}(T)$ (labelled c). Otherwise: the typing atom $\mathsf{T}(\hat{c}, \mathsf{C}(T))$ and the flattened axiom $\mathrm{Flat}_{\langle\rangle}(\hat{c}, T)$.

(D2) *Definition* $c \coloneqq t : T$. The axioms of (D1) for $c : T$, and additionally the *unfolding axiom* $\hat{c} \approx_{\langle\rangle} t$ — with $\approx_\Gamma$ the levelled equality of (Δ-λ); for T kind-level of product shape the unfolding is emitted in the pointwise form $\langle \forall \vec{x}{:}\vec{A} \rangle\, (\hat{c}\,\vec{x} \approx_{\vec{x}:\vec{A}} t\,\vec{x})$ so that no $\approx$-equation between type-level terms is ever generated (cf. Lemma 4.7).

(D3) *Inductive declaration* (notation of Definition 2.2(iii); write $\vec{A}$ for the parameters and abbreviate $\sigma'_{j,l} = \sigma_{j,l}$, all in context $\vec{A}{:}\mathsf{Type}_1$):

- typing atoms and flattened typing axioms (D1-style) for I and for each constructor c_j;
- for each j, the *injectivity axiom* $\mathsf{F}(\forall \vec{A}{:}\mathsf{Type}_1.\ \forall \vec{x}, \vec{x}'.\ c_j\,\vec{A}\,\vec{x} \doteq c_j\,\vec{A}\,\vec{x}' \to \bigwedge_l x_l \doteq x'_l)$;
- for each $i \neq j$, the *discrimination axiom* $\mathsf{F}(\forall \vec{A}{:}\mathsf{Type}_1.\ \forall \vec{x}, \vec{x}'.\ \neg\,(c_i\,\vec{A}\,\vec{x} \doteq c_j\,\vec{A}\,\vec{x}'))$;
- the *inversion axiom* $\mathsf{F}(\forall \vec{A}{:}\mathsf{Type}_1.\ \forall z{:}I\,\vec{A}.\ \bigvee_j \exists \vec{x}_j{:}\vec{\sigma}_j.\ z \doteq c_j\,\vec{A}\,\vec{x}_j)$

(the quantified bodies are the evident CIC$^-$ propositions; their F-translations expand the guards).

(D4) *Universe atom.* $\mathsf{T}(\ulcorner \mathsf{Prop} \urcorner, \ulcorner \mathsf{Type}_1 \urcorner)$. **No** axiom $\mathsf{T}(\ulcorner \mathsf{Type}_1 \urcorner, \ulcorner \mathsf{Type}_1 \urcorner)$ is emitted, in contrast to [CK18, §5.4]: that atom is false in the intended semantics, and in the stratified calculus CIC$^-$ it is never needed.

Given a finite set P of *premises* (parameter declarations $p_i : \varphi_i$ with φ_i closed propositions) and a closed goal proposition φ_G, the *translated problem* is

$$\mathrm{Transl}(P, \varphi_G) \ =\ \big(\Delta_{\mathrm{gen}} \cup \{\, \mathsf{F}(\varphi_i) \mid p_i \in P \,\},\ \mathsf{F}(\varphi_G) \big)$$

where Δ_{gen} collects (D2)–(D4) axioms for all declarations reachable from $P \cup \{\varphi_G\}$ together with all axioms emitted by the recursive calls of C.

Lemma 4.7 (No type-level equations)**.** *No axiom or conjecture of a translated problem contains an $\approx$-literal either side of which is the translation of a data type or kind-level term. All equations between such terms are rendered in the extensional form $\forall z.\, \mathsf{T}(z, \cdot) \leftrightarrow \mathsf{T}(z, \cdot)$ by the levelled equality.*

Proof. By inspection of Definitions 4.5 and 4.6: $\approx$ is produced only by (F4), where both sides are data objects by the typing rule of $\doteq$; by (Δ-λ), (Δ-case) and (D2) through the levelled equality, whose $\approx$-clause is restricted to data-object bodies; and by (Δ-case) in the scrutinee equation, whose sides are data objects. □

4.4 The induced structure $\mathfrak{M}^*$

Definition 4.8 (Standard first-order structure)**.** The structure $\mathfrak{M}^*$ has domain $\mathcal{D} \coloneqq \mathcal{U}_2$, and interprets:

- $u \mathbin{@} v$: $\mathrm{ap}(f, a) \coloneqq f(a)$ if f is a function (graph) with $a \in \mathrm{dom}(f)$, and $\emptyset$ otherwise;
- $\mathsf{T}(u, v)$: the membership relation $\{\, (a, b) \in \mathcal{D}^2 \mid a \in b \,\}$;
- $\mathsf{Pr}(u)$: $\{\, a \in \mathcal{D} \mid \emptyset \in a \,\}$;
- $\approx$: identity;
- each constant $\hat{c}$ (including lifted constants, via Definition 4.3) by $[\![c]\!] \in \mathcal{D}$; and $\ulcorner \mathsf{Prop} \urcorner \mapsto \Omega$, $\ulcorner \mathsf{Type}_1 \urcorner \mapsto \mathcal{U}_1$.

A pair (ρ, ν) of a Γ-assignment and a first-order valuation is *conforming* if $\nu(x) = \rho(x)$ for every non-proof variable x of Γ.

Lemma 4.9 (Compositionality)**.** *Let t be a Γ-term in the domain of $\mathsf{C}_\Gamma(-)$ and let (ρ, ν) be conforming. Then the value of $\mathsf{C}_\Gamma(t)$ in $\mathfrak{M}^*$ under ν equals $[\![t]\!]_\rho$.*

Proof. Induction on t. Variables and constants: by conformity and Definition 4.8; sorts: $[\![\mathsf{Prop}]\!] = \Omega$, $[\![\mathsf{Type}_1]\!] = \mathcal{U}_1$ by (M2). Application $t\,u$: by the induction hypothesis the arguments of @ evaluate to $[\![t]\!]_\rho$ and $[\![u]\!]_\rho$. Since t is not a proof, its type is a non-propositional product $\Pi x{:}A.\, B$, so by (M1) and (M4) the value $[\![t]\!]_\rho$ is a graph with domain $[\![A]\!]_\rho \ni [\![u]\!]_\rho$; hence the ap-clause applies and yields $[\![t]\!]_\rho([\![u]\!]_\rho) = [\![t\,u]\!]_\rho$ by (M5). Lifted clauses (C3)–(C6): the translation is $\ulcorner t \urcorner \vec{y}$, whose value is $\mathrm{ap}(\cdots \mathrm{ap}([\![\ell]\!], \rho(y_1)) \cdots, \rho(y_m))$ with $\ell \coloneqq \lambda \vec{y}{:}\vec{\rho}.\, t$. Each application step is within the successive domains by (M1) (the y_i-values conform), so by (M5), (M6) and (M7) the value equals $[\![(\lambda \vec{y}.\, t)\, \vec{y}]\!]_\rho = [\![t]\!]_\rho$. □

4.5 The truth lemma and base soundness

Theorem 4.10 (Truth lemma)**.** *Let φ be a Γ-proposition and (ρ, ν) conforming. Then*

$$\mathfrak{M}^*, \nu \models \mathsf{F}_\Gamma(\varphi) \qquad \Longleftrightarrow \qquad [\![\varphi]\!]_\rho = \{\emptyset\}.$$

Moreover every axiom emitted by the translation of any term occurring in the problem is true in $\mathfrak{M}^*$.

Proof. Both statements are proved simultaneously by induction on the size of the translated term (the axioms emitted for a lifted subterm mention only translations of terms of smaller or equal size in extended contexts; we make the measure precise below). Throughout, "true" means true in $\mathfrak{M}^*$ under the ambient valuation, and we use Lemma 4.9 silently for all C-images.

Measure. Define the weight of a term as its size, and order proof obligations as follows: the truth-lemma obligation for φ depends on truth-lemma obligations for immediate subformulas of φ (smaller) and on nothing else; the axiom-truth obligations for a lifted constant $\ulcorner t \urcorner_{\vec{y}:\vec{\rho}}$ depend on truth-lemma obligations for terms of size at most that of t's components (b, branch bodies, A, B), each smaller than t. So the induction is well-founded.

Truth lemma, case (F1) $\varphi = \Pi x{:}A.\, \psi$ with A a proposition, $x \notin \mathrm{FV}(\psi)$. By (M4), $[\![\varphi]\!]_\rho = \{\emptyset\}$ iff ($[\![A]\!]_\rho = \emptyset$ or $[\![\psi]\!]_\rho = \{\emptyset\}$): indeed $[\![A]\!]_\rho \in \Omega$, and the trace-product condition "$\forall a \in [\![A]\!]_\rho.\ [\![\psi]\!]_{\rho[x \mapsto a]} = \{\emptyset\}$" is vacuous when $[\![A]\!]_\rho = \emptyset$ and equivalent to $[\![\psi]\!]_\rho = \{\emptyset\}$ when $[\![A]\!]_\rho = \{\emptyset\}$ (as $x \notin \mathrm{FV}(\psi)$, by (M7) the value does not depend on a). Classically, the right-hand side is exactly the truth condition of $\mathsf{F}(A) \to \mathsf{F}(\psi)$ by the induction hypothesis for A and ψ.

Case (F2) $\varphi = \Pi x{:}A.\,\psi$, A not a proposition. The translation is $\forall x.\ \mathsf{T}(x, \mathsf{C}_\Gamma(A)) \to \mathsf{F}_{\Gamma,x:A}(\psi)$. Since $\mathsf{T}^{\mathfrak{M}^*}$ is membership and $\mathsf{C}_\Gamma(A)$ evaluates to $[\![A]\!]_\rho$, the translated formula is true iff for every $d \in [\![A]\!]_\rho$ the formula $\mathsf{F}_{\Gamma,x:A}(\psi)$ is true under $\nu[x \mapsto d]$. For each such d, the pair $(\rho[x \mapsto d], \nu[x \mapsto d])$ is conforming for $\Gamma, x{:}A$, and the induction hypothesis turns the condition into: for every $d \in [\![A]\!]_\rho$, $[\![\psi]\!]_{\rho[x\mapsto d]} = \{\emptyset\}$ — which by (M4) is exactly $[\![\varphi]\!]_\rho = \{\emptyset\}$. Note this equivalence is *exact* in both directions: the guard carves out precisely the set $[\![A]\!]_\rho$ from the domain $\mathcal{D}$. This is the point of using membership guards in hypothesis positions.

Case (F3), (F4), (F5): immediate from (M8), the induction hypothesis, and (for F4) Lemma 4.9; for (F5) note again that the guard in $\exists x.\ \mathsf{T}(x, \mathsf{C}_\Gamma(A)) \wedge \cdots$ ranges exactly over $[\![A]\!]_\rho$.

Case (F6) φ atomic. $\mathsf{F}_\Gamma(\varphi) = \mathsf{Pr}(\mathsf{C}_\Gamma(\varphi))$, and $\mathsf{C}_\Gamma(\varphi)$ evaluates to $[\![\varphi]\!]_\rho \in \Omega$ (by (M1)). Now $\mathsf{Pr}^{\mathfrak{M}^*}(a)$ iff $\emptyset \in a$, and for $a \in \Omega$ this is equivalent to $a = \{\emptyset\}$.

Axiom truth. We verify each emitted axiom family.

(Δ-prop): under any values $\vec{d}$ of $\vec{y}$ passing their guards, the pair (ρ, ν) with $\rho = [\vec{y} \mapsto \vec{d}]$ is conforming; the left side $\mathsf{Pr}(\ulcorner t \urcorner\, \vec{y})$ holds iff $\emptyset \in [\![t]\!]_\rho$ iff $[\![t]\!]_\rho = \{\emptyset\}$, and the right side iff the same by the induction hypothesis (truth lemma for t, smaller than the lifting trigger). Values of $\vec{y}$ *failing* their guards make the guarded closure vacuously true.

(Δ-app): let $\vec{d}$ pass the guards, let e satisfy $\mathsf{T}(z, \ulcorner t \urcorner\, \vec{y})$, i.e. $e \in [\![\Pi x{:}A.\,B]\!]_\rho$, and let $a \in [\![A]\!]_\rho$. By (M4), e is a graph with domain $[\![A]\!]_\rho$ and $e(a) \in [\![B]\!]_{\rho[x\mapsto a]}$; by the ap-clause, $z\,@\,x$ evaluates to $e(a)$, and $\mathsf{C}_{\Gamma,x:A}(B)$ evaluates to $[\![B]\!]_{\rho[x\mapsto a]}$ by Lemma 4.9. So the conclusion atom is membership of $e(a)$ in exactly that set: true.

(Δ-λ): under guard-passing values, both sides of the levelled equality evaluate compatibly: $\ulcorner t \urcorner\, \vec{y}\, \vec{x}$ evaluates, as in Lemma 4.9, to $[\![b]\!]_\rho$; the right side is $\mathsf{C}(b)$ (or $\mathsf{F}(b)$ under Pr, or the T-extension of $\mathsf{C}(b)$), which denotes/holds-of the same set by Lemma 4.9 (respectively by the induction hypothesis for the Pr-clause; for the type-level clause both T-atoms test membership in the same set, so the $\forall z$-equivalence is true).

(Δ-ty): the typing atom asserts $[\![\ell\, \vec{y}]\!] \in [\![T]\!]_\rho$ — an instance of (M1). The flattened axiom: by induction on T's product prefix, repeatedly applying (M4) exactly as in the (Δ-app) case: guard-passing arguments lie in the successive domains, applications stay in the successive codomains, and the final atom is membership of the fully applied value in the final codomain — true by (M1) and (M4).

(Δ-case): let $\vec{d}$ pass the guards of $\vec{y}$; write ρ for the induced assignment. By (M1), $[\![u]\!]_\rho \in [\![I]\!]([\![\vec{\tau}]\!]_\rho)$, so by (M9)(d) there are unique j and arguments $\vec{e}$ (in the appropriate denotations, (M9)(a)) with $[\![u]\!]_\rho = [\![c_j]\!]([\![\vec{\tau}]\!]_\rho)(\vec{e})$. Choose the j-th disjunct and witnesses $\vec{e}$: the guards on $\vec{x}_j$ hold by construction; the scrutinee equation holds by Lemma 4.9; and the levelled equality between $\ulcorner t \urcorner\, \vec{y}$ and $u_j\, \vec{x}_j$ holds because by (M10) $[\![t]\!]_\rho = [\![u_j]\!]_\rho(\vec{e}) = [\![u_j\, \vec{x}_j]\!]_{\rho[\vec{x}_j\mapsto\vec{e}]}$, and the levelled equality's truth then follows exactly as in the (Δ-λ) case.

(D1)/(D2): typing atoms and flattened axioms as in (Δ-ty); premise formulas are handled by Theorem 4.11 below; unfolding axioms: by (M3) $[\![c]\!] = [\![t]\!]$, and the levelled equality between two names of the same set is true as in (Δ-λ).

(D3): each of the injectivity, discrimination and inversion axioms is $\mathsf{F}(\chi)$ for a closed CIC$^-$ proposition χ; by the truth-lemma part (already available for χ, which is not larger than the trigger — these axioms are emitted at declaration translation time, and we simply order them after all truth-lemma obligations for their own subterms), it suffices that χ is valid. Validity of injectivity is (M9)(b); of discrimination, (M9)(c); of inversion, (M9)(d) (every element of $[\![I]\!](\vec{a})$ is a constructor value with arguments in the required denotations).

(D4): $\Omega \in \mathcal{U}_1$: true, as $\Omega \in \mathcal{U}_1$ by Definition 3.1. □

Theorem 4.11 (Base soundness)**.** *Let E be admissible with standard model $\mathcal{M}$, let P be premises whose statements φ_i are valid in $\mathcal{M}$ (e.g. provable, cf. Proposition 3.3), and let φ_G*

be a closed goal proposition. If

$$\Delta_{\mathrm{gen}} \cup \{\, \mathsf{F}(\varphi_i) \mid i \,\} \;\vdash_{\mathrm{c}}\; \mathsf{F}(\varphi_G),$$

then φ_G is valid in $\mathcal{M}$. In particular the axiom set $\Delta_{\mathrm{gen}} \cup \{\mathsf{F}(\varphi_i)\}_i$ is consistent.

Proof. By Theorem 4.10, $\mathfrak{M}^*$ satisfies Δ_{gen}, and satisfies each $\mathsf{F}(\varphi_i)$ since φ_i is valid. By soundness of classical provability, $\mathfrak{M}^* \models \mathsf{F}(\varphi_G)$. By the converse direction of Theorem 4.10, $[\![\varphi_G]\!] = \{\emptyset\}$. Consistency: $\mathfrak{M}^*$ is a model of the axiom set. □

We record the invariant that the rest of the paper preserves.

Definition 4.12 (Soundness invariant)**.** A first-order problem (Φ, γ) is *$\mathcal{M}$-sound* if there exists an expansion $\mathfrak{M}$ of $\mathfrak{M}^*$ by interpretations of finitely many fresh symbols such that (i) $\mathfrak{M} \models \Phi$, and (ii) $\mathfrak{M} \models \gamma$ implies that φ_G is valid in $\mathcal{M}$, where φ_G is the pending goal. If (Φ, γ) is $\mathcal{M}$-sound and $\Phi \vdash_{\mathrm{c}} \gamma$, then φ_G is valid. Theorem 4.11 states that $\mathrm{Transl}(P, \varphi_G)$ is $\mathcal{M}$-sound (with $\mathfrak{M} = \mathfrak{M}^*$ and (ii) supplied by Theorem 4.10).

4.6 Discussion: design choices and the implemented translation

Remark 4.13 (The polarity discipline). Typing information appears in two polarities. In *hypothesis* position — the guard of a universally quantified premise variable — our translation always uses the membership guard $\mathsf{T}(x, \mathsf{C}(A))$. In *conclusion* position — typing axioms for constants and lifted functions — it emits both the membership atom and the pointwise Flat axiom. Both conclusion forms are true in $\mathfrak{M}^*$ (Theorem 4.10); the pointwise form in hypothesis position would *not* be: the formula $\forall f.(\forall x.\, \mathsf{T}(x, \hat{A}) \to \mathsf{T}(f\,@\,x, \hat{B})) \to \mathsf{F}(\psi)$ quantifies also over graphs whose domain strictly contains $[\![A]\!]$; such junk elements satisfy the pointwise hypothesis but need not satisfy ψ's translation, so the translated premise could be false in $\mathfrak{M}^*$ even when the CIC$^-$ premise is valid. For the same reason a *bidirectional* characterisation axiom $\forall z.\, \mathsf{T}(z, \ulcorner \Pi x{:}A.B \urcorner) \leftrightarrow (\forall x. \ldots)$ — which the CoqHammer implementation emits for lifted type constants (`opt_type_lifting`) — is unsound for $\mathfrak{M}^*$: its right-to-left direction asserts that the pointwise class is exactly $[\![\Pi x{:}A.\, B]\!]$, which fails in any membership interpretation. Our (Δ-app) keeps only the sound elimination direction, exactly as the axiom family Δ_{G_1} of [Cza18, Def. 45] does. The translation of [CK18, §5.2] uses the pointwise form (via its function G) also in hypothesis positions; that choice is one of the reasons the implemented translation "is neither sound nor complete" in the strict sense [CK18, §5.6], while its guarded core in the discipline used here is validated both proof-theoretically [Cza18] and model-theoretically (Theorem 4.10).

Remark 4.14 (Functional extensionality). In $\mathfrak{M}^*$, elements of $[\![A \to B]\!]$ are graphs with domain exactly $[\![A]\!]$, so pointwise equal elements are equal: the translation of the functional extensionality axiom is *true* in $\mathfrak{M}^*$, and may soundly appear among the premises. The inconsistency discussed in [CK18, §5.6] — functional extensionality plus Coq equality mapped to $\approx$ — arises only in combination with the *omission of guards* on the free and bound variables of lifted abstractions, which the implementation performs by default and our base translation does not. Once guards are kept, the extensionality instance forced by (Δ-λ) applies only to guard-passing arguments, and truth in $\mathfrak{M}^*$ is restored. The systematic study of which guards may be dropped is exactly the subject of Section 7; the erasure criterion proved there authorises dropping guards only at types where no collapse of the kind exhibited in Proposition 7.19 can occur.

Remark 4.15 (Incompleteness). The translation is deliberately incomplete, and no claim of completeness is made anywhere in this paper. Known sources include the absence of

ξ/extensionality reasoning for lifted constants ([Cza18, Example 51]) and the loss of higher-order instantiation. Completeness statements below (Propositions 5.7 and 7.3 and Theorem 6.4) are always *relative*: transformation X does not lose first-order proofs that existed before X.

Example 4.16. Let E contain nat (constructors O, S), the parametric list (constructors $\mathsf{nil} : \Pi A{:}\mathsf{Type}_1.\, \mathsf{list}\, A$ and $\mathsf{cons} : \Pi A{:}\mathsf{Type}_1.\, A \to \mathsf{list}\, A \to \mathsf{list}\, A$), and a predicate $\mathsf{In} : \Pi A{:}\mathsf{Type}_1.\, A \to \mathsf{list}\, A \to \mathsf{Prop}$. The premise

$$\varphi \;=\; \Pi A{:}\mathsf{Type}_1.\; \Pi x{:}A.\; \Pi l{:}\mathsf{list}\, A.\; \mathsf{In}\, A\, x\, (\mathsf{cons}\, A\, x\, l)$$

translates to

$$\forall A.\; \mathsf{T}(A, \ulcorner \mathsf{Type}_1 \urcorner) \to \forall x.\; \mathsf{T}(x, A) \to \forall l.\; \mathsf{T}(l, \widehat{\mathsf{list}} \mathbin{@} A) \to \mathsf{Pr}\big(\widehat{\mathsf{In}} \mathbin{@} A \mathbin{@} x \mathbin{@} (\widehat{\mathsf{cons}} \mathbin{@} A \mathbin{@} x \mathbin{@} l)\big).$$

Every literal carries the type variable A; the inversion axiom for list is likewise generic in A. Section 5 is about replacing such formulas by instances at the ground types at which they are used, e.g. $A := \mathsf{nat}$; Section 6 then turns $\mathsf{T}(l, \widehat{\mathsf{list}} \mathbin{@} \widehat{\mathsf{nat}})$ into $\mathsf{P}_{\mathsf{list\,nat}}(l)$ and flattens the applications; and Section 7 deletes the guards that monotonicity renders redundant.

5 Heuristic monomorphisation

Monomorphisation replaces (or supplements) premises that quantify over types by finitely many *instances* at ground types. We formulate it as an operation on CIC$^-$ propositions — instantiate, normalise, retranslate — rather than as an operation on the generated first-order formulas. The choice matters: after instantiation, $\beta\iota\delta$-redexes disappear (Example 5.9), the type-directed translation re-runs on the instance and produces structurally better formulas, and the soundness argument is a two-line application of the semantics. A purely first-order instantiation of the translated generic formula would have to reproduce these effects through bridging axioms, which the design constraint of a *shallow* translation — every transformation is a per-occurrence formula manipulation, with no mediating axiomatic theory — rules out.

Throughout this section, fix an admissible environment E and a standard model $\mathcal{M}$.

5.1 Instances of premises

Definition 5.1 (Prefix; instantiation)**.** Let φ be a closed proposition in $\beta\iota\delta$-normal form, written with maximal product prefix

$$\varphi \;=\; \Pi x_1{:}B_1.\; \cdots\; \Pi x_n{:}B_n.\; \psi \qquad (\psi \text{ not a product}).$$

An *instantiation* of φ is a partial function θ on $\{x_1, \ldots, x_n\}$ such that, processing positions in order, each $x_i \in \mathrm{dom}\theta$ is mapped to a *closed* $\beta\iota\delta$-normal term $\theta(x_i)$ with $\vdash \theta(x_i) : B_i\theta^{<i}$, where $\theta^{<i}$ denotes the simultaneous substitution of the already-processed values. The *instance* is

$$\varphi \cdot \theta \;:=\; \mathrm{nf}\Big(\Pi_{i \notin \mathrm{dom}\theta}\, x_i : B_i\theta.\; \psi\theta\Big),$$

the $\beta\iota\delta$-normal form of the result of deleting the instantiated binders and substituting throughout (no capture can occur: the substituted terms are closed). We call θ a *type instantiation* if every $\theta(x_i)$ is a data type (then necessarily $B_i\theta^{<i} = \mathsf{Type}_1$); the general case additionally instantiates data and kind-level binders by closed terms and is used for higher-order instance generation (Remark 5.10). A closed proposition is *monomorphic* if Type_1 does not occur in it.

Lemma 5.2 (Instances are well-formed)**.** *$\varphi \cdot \theta$ is a closed proposition, i.e. $\vdash \varphi \cdot \theta : \mathsf{Prop}$.*

Proof. By induction on n. If $x_1 \notin \mathrm{dom}\theta$, apply the induction hypothesis to $\psi' = \Pi x_2 \ldots \psi$ in the context $x_1{:}B_1$ (the definition and the claim relativise to a context of uninstantiated binders in the obvious way) and rebuild the product with the rule that formed it. If $x_1 \in \mathrm{dom}\theta$ with value τ: by generation, $x_1{:}B_1 \vdash \Pi x_2 \ldots \psi : \mathsf{Prop}$; by the substitution property Proposition 2.6(2) (in the form of Lemma 2.9 when $B_1 = \mathsf{Type}_1$; in general by Proposition 2.6(2) directly), $\vdash (\Pi x_2 \ldots \psi)[\tau/x_1] : \mathsf{Prop}$, and we proceed by the induction hypothesis. Normalisation preserves typing by subject reduction Proposition 2.6(3), and the normal form exists and is unique by Proposition 2.6(1),(5). □

Lemma 5.3 (Semantic instantiation)**.** *If φ is valid in $\mathcal{M}$ then so is $\varphi \cdot \theta$.*

Proof. By induction on n; we show the stronger contextual statement: if Γ-proposition $\Pi x{:}B.\,\chi$ has $[\![\Pi x{:}B.\,\chi]\!]_\rho = \{\emptyset\}$ and $\vdash \tau : B$ is closed, then $[\![\chi[\tau/x]]\!]_\rho = \{\emptyset\}$; and if x is not instantiated, validity passes to all $[\![\chi]\!]_{\rho[x\mapsto a]}$, $a \in [\![B]\!]_\rho$, and is recovered for the rebuilt product. Both parts are immediate from (M4): validity of the product means $[\![\chi]\!]_{\rho[x\mapsto a]} = \{\emptyset\}$ for *every* $a \in [\![B]\!]_\rho$; for the instantiated case take $a = [\![\tau]\!]$, which lies in $[\![B]\!]_\rho$ by (M1), and use (M7) to identify $[\![\chi[\tau/x]]\!]_\rho = [\![\chi]\!]_{\rho[x\mapsto[\![\tau]\!]]}$. Finally, normalisation does not change denotations by (M6). □

Instances are also *derivable* from their premises inside the type theory, which matters for the reconstruction phase: any proof using an instance can be replayed from the original premise.

Proposition 5.4 (Syntactic instantiation)**.** *If $\vdash p : \varphi$ then there is a term p^θ with $\vdash p^\theta : \varphi \cdot \theta$.*

Proof. By induction on the prefix, define p^θ: for $x_1 \in \mathrm{dom}\theta$ take $(p\,\theta(x_1))^{\theta'}$ (application rule, then the induction hypothesis for the substituted suffix); for $x_1 \notin \mathrm{dom}\theta$ take $\lambda x_1{:}B_1\theta.\ (p\,x_1)^\theta$ (abstraction rule). The resulting type is convertible to $\varphi\cdot\theta$, and the conversion rule concludes. □

5.2 Monomorphisation of problems

Definition 5.5 (Monomorphisation)**.** Let (P, φ_G) be a problem (Definition 4.6). Recall that the emitted inductive axioms (D3) are the F-translations of closed CIC$^-$ propositions; we treat these propositions as (implicit, valid) premises, so that instantiation applies to them uniformly. A *monomorphisation* is a family $\Theta = (\Theta_p)_p$ assigning to each premise p (explicit or implicit) a finite set of instantiations of its statement φ_p. It induces two premise sets:

$$\mathrm{Mono}_\Theta^{\mathrm{keep}}(P) \;=\; P \;\cup\; \{\, \varphi_p \cdot \theta \mid p,\ \theta \in \Theta_p \,\},$$
$$\mathrm{Mono}_\Theta^{\mathrm{drop}}(P) \;=\; \{\, \varphi_p \cdot \theta \mid p,\ \theta \in \Theta_p,\ \varphi_p \cdot \theta \text{ monomorphic} \,\} \;\cup\; \{\, \varphi_p \mid p,\ \varphi_p \text{ monomorphic} \,\}.$$

The monomorphised problems are $\mathrm{Transl}(\mathrm{Mono}_\Theta(P), \varphi_G)$ for either policy.

Theorem 5.6 (Soundness of monomorphisation)**.** *For every monomorphisation Θ and either policy, the monomorphised problem is $\mathcal{M}$-sound (Definition 4.12). Consequently, if the ATP proves the translated goal from the monomorphised premises, the goal is valid in $\mathcal{M}$.*

Proof. Every element of $\mathrm{Mono}_\Theta(P)$ is either a premise statement or an instance of one, hence valid in $\mathcal{M}$ by hypothesis and Lemma 5.3; the implicit (D3) premises are valid by (M9) as in Theorem 4.10. Theorem 4.11 applied to the premise set $\mathrm{Mono}_\Theta(P)$ gives the claim. (The axiom store grows — translating instances lifts new subterms — but Theorem 4.10 covers all emitted axioms.) □

Note what this theorem does *not* depend on: how Θ was found. Any instance-selection heuristic whatsoever — including an unsound-looking syntactic matcher — yields a sound pipeline, because soundness is carried entirely by Lemmas 5.2 and 5.3. Heuristics affect only effectiveness. This modularity is the main payoff of formulating monomorphisation at the CIC$^-$ level.

Proposition 5.7 (Conservativity of *keep*)**.** *Every first-order proof available before monomorphisation remains available after* $\mathrm{Mono}_\Theta^{\mathrm{keep}}$*: if* $\mathrm{Transl}(P, \varphi_G) = (\Phi, \gamma)$ *proves* γ*, then the keep-policy problem* (Φ', γ) *also proves* γ*.*

Proof. Translation is monotone in the premise set and idempotent on the axiom store (lifted constants and their axioms are indexed by α-classes, Definition 4.3), so $\Phi \subseteq \Phi'$, and classical provability is monotone. □

Proposition 5.8 (Incompleteness of *drop*)**.** *There is a problem* (P, φ_G) *and a monomorphisation* Θ *such that* $\mathrm{Transl}(P, \varphi_G)$ *proves its conjecture but the drop-policy problem does not.*

Proof. Let E declare data types σ, τ with closed inhabitants $a : \sigma$, and $f : \Pi A{:}\mathsf{Type}_1.\, A \to A$, and take the premise $\varphi_p = \Pi A{:}\mathsf{Type}_1.\, \Pi x{:}A.\ f\, A\, x \doteq_A x$ with goal $\varphi_G = f\, \sigma\, a \doteq_\sigma a$, and $\Theta_p = \{[A \mapsto \tau]\}$. The generic problem proves the goal (instantiate the guarded universal with $\hat\sigma$ and $\hat a$; the guards $\mathsf{T}(\hat\sigma, \ulcorner \mathsf{Type}_1 \urcorner)$ and $\mathsf{T}(\hat a, \hat\sigma)$ are typing atoms of the translation). For the drop-policy problem, only the τ-instance is present; a countermodel is obtained from $\mathfrak{M}^*$ by reinterpreting ap at the point $([\![f]\!], [\![\sigma]\!])$: replace $[\![f]\!]$'s value at σ by a non-identity function g on $[\![\sigma]\!]$ with $g([\![a]\!]) \neq [\![a]\!]$ (possible whenever $|[\![\sigma]\!]| \geq 2$; formally, reinterpret the constant $\hat f$ by a graph agreeing with $[\![f]\!]$ except at argument $[\![\sigma]\!]$). All axioms of the drop-policy problem mention f only at τ or generically through typing atoms that remain true (the modified graph still maps $[\![\sigma]\!]$ into $[\![\sigma]\!] \to [\![\sigma]\!]$), while the conjecture is now false. □

Example 5.9 (Why instances are built before translation)**.** Consider the polymorphic inversion premise for the inductive predicate $\mathsf{Forall} : \Pi A{:}\mathsf{Type}_1.\, (A \to \mathsf{Prop}) \to \mathsf{list}\, A \to \mathsf{Prop}$:

$$\varphi \;=\; \forall A{:}\mathsf{Type}_1.\, \forall P{:}A \to \mathsf{Prop}.\, \forall l{:}\mathsf{list}\, A.\, \mathsf{Forall}\, A\, P\, l \leftrightarrow \big(l \doteq \mathsf{nil}\, A \vee \exists x{:}A.\, \exists l'{:}\mathsf{list}\, A.\ (P\, x \wedge \cdots)\big).$$

Its translation contains the higher-order application pattern $\mathsf{Pr}(P \mathbin{@} x)$ with P a quantified variable, and guards $\mathsf{T}(P, \mathsf{C}(A \to \mathsf{Prop}))$ through a lifted type constant — both notoriously hard for ATPs. Suppose the goal concerns $\mathsf{Forall}\, \mathsf{nat}\, (\lambda x{:}\mathsf{nat}.\, \mathsf{lt}\, \mathsf{O}\, x)\, l$. The occurrence yields the instantiation $\theta = [A \mapsto \mathsf{nat},\ P \mapsto \lambda x.\, \mathsf{lt}\, \mathsf{O}\, x]$ (a general instantiation in the sense of Definition 5.1); in $\varphi \cdot \theta$ the redex $(\lambda x.\, \mathsf{lt}\, \mathsf{O}\, x)\, x$ is *normalised away*, so the instance translates to a first-order formula whose corresponding subformula is the direct atom $\mathsf{Pr}(\widehat{\mathsf{lt}} \mathbin{@} \hat{\mathsf{O}} \mathbin{@} x)$ — first-order, guard-light, and after Section 6 simply $\mathsf{lt}(\mathsf{O}, x)$. No first-order manipulation of the translated generic formula can achieve this: the β-step happens in the type theory.

Remark 5.10 (Higher-order instances). Lemmas 5.2 and 5.3 and Proposition 5.4 were deliberately proved for *general* instantiations, not just type instantiations. Instantiating a higher-order premise variable $P : A \to \mathsf{Prop}$ with a closed abstraction found at an occurrence (as in Example 5.9) is therefore covered by the same soundness theorem. The practical iteration scheme below applies unchanged; only the occurrence-matching heuristic differs. This is the analogue of Lean-auto's instantiation of "constant instances with dependent arguments" [QCBA25].

5.3 Instance discovery and bounds

Soundness being heuristic-independent, we specify the discovery algorithm only to the degree needed to state termination and cost bounds. The scheme is the standard goal-seeded saturation of Sledgehammer [BBPS16, §7.1], transplanted to constant instances in the sense of Definition 5.1.

Definition 5.11 (Occurrence extraction; saturation)**.** For a normal term t, let $\mathrm{TA}(t)$ be the finite set of pairs $(c, \vec{\tau})$ such that an application $c\,\tau_1 \cdots \tau_j$ occurs in t where c is a constant whose type has a product prefix beginning with j binders of domain Type_1 and each τ_i is closed and normal. Given a problem (P, φ_G) and parameters $K, \Delta_{\mathrm{tot}}, \Delta_c \in \mathbb{N}$, the saturation computes, for $r = 0, 1, \ldots, K$: the *active instance pool* W_r ($W_0 = \mathrm{TA}(\varphi_G)$), and for each generic premise p the instantiations obtained by matching the type-variable prefix of φ_p against elements of W_r occurring in φ_p's own constant applications; the resulting instances contribute their TA to W_{r+1}. Generation stops after round K, or as soon as the number of instances generated for one premise exceeds Δ_c or the total exceeds Δ_{tot} (excess instances are discarded by a fixed fair strategy).

Proposition 5.12. *The saturation terminates, produces at most Δ_{tot} instances, and every produced θ is an instantiation in the sense of Definition 5.1; hence Theorem 5.6 applies.*

Proof. Termination and the cardinality bound are immediate from the caps. Well-formedness of the produced θ: the matched type arguments are closed normal subterms of well-typed terms at Type_1-typed positions, so each binding satisfies the typing requirement of Definition 5.1 (formally: a subterm occurrence of a well-typed term is well-typed in its local context; a closed subterm at a position of type Type_1 is a closed data type). □

Sledgehammer operates with $K = 3$ rounds and a total cap of 200 new formulas by default, values twice validated experimentally [BBPS16, DVBW22]; Lean-auto uses a comparable global cap [QCBA25]. We adopt them as defaults.

5.4 No complete instantiation strategy exists

Heuristic monomorphisation with the *drop* policy is incomplete (Proposition 5.8); with the *keep* policy it is conservative but the generic formulas remain a burden. One might hope for a clever computable strategy producing, for each problem, a finite instance set that is *complete* — one that loses nothing. For polymorphic first-order logic, Bobot and Paskevich proved this hope vain [BP11, Thm. 1]. We now prove the analogous result for CIC^{-}, adapting their reduction (from ground reducibility in combinatory logic, with combinator terms reflected at the type level) to the dependent-type-theoretic setting. The notion of consequence is semantic, matching our soundness framework.

Definition 5.13 (Consequence)**.** Let E be an environment and P a finite set of closed propositions, φ a closed proposition. Write $P \models_E \varphi$ if for every standard model $\mathcal{M}$ of E in which all members of P are valid, φ is valid in $\mathcal{M}$. (Standard models differ in their interpretation of parameter constants; (M9) pins inductive types only up to the listed properties.)

By Lemma 5.3, instances are consequences: $\{\, \varphi_p \cdot \theta \mid \ldots \,\} \models_E \varphi$ implies $P \models_E \varphi$. A *complete instantiation strategy* would be a computable converse:

Theorem 5.14 (Undecidability of complete monomorphisation)**.** *There is no algorithm that, given an environment E, a finite premise set P and a closed goal φ, computes a finite monomorphisation Θ such that*

$$P \models_E \varphi \quad\Longrightarrow\quad \{\, \varphi_p \cdot \theta \mid p,\ \theta \in \Theta_p \,\} \models_E \varphi.$$

Proof. We construct a family of problems encoding weak reduction in combinatory logic (CL). CL terms are $w ::= K \mid S \mid w\,w$; weak reduction is generated by $K\,x\,y \to x$, $S\,x\,y\,z \to x\,z\,(y\,z)$ and congruence, and $\to^*$ is its reflexive–transitive closure. The relation $w \to^* w'$ between ground CL terms is undecidable: by combinatory completeness [Bar84] there is, for every partial recursive f, a CL term F with $F\,\overline{n} \to^* \overline{f(n)}$ for all n in the domain of f (numerals $\overline{n}$ in normal form), and $F\,\overline{n} \to^* \overline{0}$ is then equivalent to $f(n) = 0$, which for suitable f is Σ_1-complete.

The environment E_{CL}. Declare data inductives: K_0 with one 0-ary constructor k_0; S_0 with *two* 0-ary constructors $\mathsf{s}_0, \mathsf{s}_1$ (the second is a tag ensuring $[\![\mathsf{K}_0]\!] \neq [\![\mathsf{S}_0]\!]$ in every model, by (M9)(b–d): one denotation is a singleton, the other a two-element set); and A with parameters $\alpha, \beta{:}\mathsf{Type}_1$ and one constructor $\mathsf{a}_0 : \Pi\alpha\beta{:}\mathsf{Type}_1.\ \alpha \to \beta \to \mathsf{A}\,\alpha\,\beta$. Declare a parameter $R : \Pi\alpha\beta{:}\mathsf{Type}_1.\ \alpha \to \beta \to \mathsf{Prop}$. Reflect CL terms as closed types and canonical points:

$$\ulcorner K \urcorner = \mathsf{K}_0, \quad \ulcorner S \urcorner = \mathsf{S}_0, \quad \ulcorner w\,w' \urcorner = \mathsf{A}\,\ulcorner w \urcorner\,\ulcorner w' \urcorner;$$
$$p_K = \mathsf{k}_0, \quad p_S = \mathsf{s}_0, \quad p_{w\,w'} = \mathsf{a}_0\,\ulcorner w \urcorner\,\ulcorner w' \urcorner\,p_w\,p_{w'}.$$

The premise set P_{CL} consists of the following closed propositions (all prefixes over Type_1; element variables typed by the type variables):

$$\forall\alpha.\ \forall u{:}\alpha.\ R\,\alpha\,\alpha\,u\,u \tag{refl}$$
$$\forall\alpha\beta\gamma.\ \forall u{:}\alpha\,v{:}\beta\,w{:}\gamma.\ R\,\alpha\,\beta\,u\,v \to R\,\beta\,\gamma\,v\,w \to R\,\alpha\,\gamma\,u\,w \tag{trans}$$
$$\forall\alpha\beta\gamma.\ \forall u{:}\alpha\,v{:}\beta\,w{:}\gamma.\ R\,\alpha\,\beta\,u\,v \to R\,(\mathsf{A}\alpha\gamma)\,(\mathsf{A}\beta\gamma)\,(\mathsf{a}_0\alpha\gamma\,u\,w)\,(\mathsf{a}_0\beta\gamma\,v\,w) \tag{congL}$$
$$\forall\alpha\beta\gamma.\ \forall u{:}\alpha\,v{:}\beta\,w{:}\gamma.\ R\,\alpha\,\beta\,u\,v \to R\,(\mathsf{A}\gamma\alpha)\,(\mathsf{A}\gamma\beta)\,(\mathsf{a}_0\gamma\alpha\,w\,u)\,(\mathsf{a}_0\gamma\beta\,w\,v) \tag{congR}$$
$$\forall\alpha\beta.\ \forall u{:}\alpha\,v{:}\beta.\ R\,\big(\mathsf{A}(\mathsf{A}\mathsf{K}_0\alpha)\beta\big)\,\alpha\,\big(\mathsf{a}_0\,(\mathsf{A}\mathsf{K}_0\alpha)\,\beta\,(\mathsf{a}_0\mathsf{K}_0\alpha\,\mathsf{k}_0\,u)\,v\big)\,u \tag{redK}$$
$$\forall\alpha\beta\gamma.\ \forall u{:}\alpha\,v{:}\beta\,w{:}\gamma.\ R\,\big(\mathsf{A}(\mathsf{A}(\mathsf{A}\mathsf{S}_0\alpha)\beta)\gamma\big)\,\big(\mathsf{A}(\mathsf{A}\alpha\gamma)(\mathsf{A}\beta\gamma)\big)\,t_S\,t'_S \tag{redS}$$

where t_S and $t'_S \coloneqq \mathsf{a}_0\,(\mathsf{A}\alpha\gamma)(\mathsf{A}\beta\gamma)\,(\mathsf{a}_0\alpha\gamma\,u\,w)\,(\mathsf{a}_0\beta\gamma\,v\,w)$ are the evident points of the redex and reduct types (both built from $\mathsf{a}_0, \mathsf{s}_0, u, v, w$). The goal for ground CL terms w, w' is $g(w, w') \coloneqq R\,\ulcorner w \urcorner\,\ulcorner w' \urcorner\,p_w\,p_{w'}$.

Claim 1: $P_{\mathrm{CL}} \models_E g(w, w')$ iff $w \to^* w'$.

($\Leftarrow$) Let $\mathcal{M}$ be any standard model validating P_{CL}. By induction on the derivation of $w \to^* w'$: each single step and each closure rule corresponds to one premise; by Lemma 5.3 the premise may be semantically instantiated at the reflected types and canonical points of the terms involved, and validity of an implication together with validity of its antecedents yields validity of its consequent ((M4), (M8)). The base facts are (redK), (redS), (refl); the guards are satisfied because $[\![p_v]\!] \in [\![\ulcorner v \urcorner]\!]$ by (M1).

($\Rightarrow$) Suppose $w \not\to^* w'$; we exhibit a standard model validating P_{CL} but not $g(w, w')$. Take the *canonical tagged model*: constructor values are tuples tagged by the constructor name, e.g. $[\![\mathsf{a}_0]\!](a, b)(x, y) = (\mathsf{a0}, a, b, x, y)$ (with $\mathsf{a0}$ a fixed set distinct from the other tags); then (M9) holds, and for every set d we can define a syntax tree $\mathrm{tr}(d)$ over the alphabet $\{K, S\} \cup \{\text{atoms}\}$ by $\in$-recursion: $\mathrm{tr}(d) = K$ if $d = [\![\mathsf{K}_0]\!]$; $\mathrm{tr}(d) = S$ if $d = [\![\mathsf{S}_0]\!]$; $\mathrm{tr}(d) = \mathrm{tr}(a) \cdot \mathrm{tr}(b)$ if $d = [\![\mathsf{A}]\!](a, b)$ for (necessarily unique, by the tagging) a, b; and $\mathrm{tr}(d) = d$ (an atom) otherwise. Uniqueness of the decomposition holds because $[\![\mathsf{A}]\!](a, b)$ is the set of tagged tuples $(\mathsf{a0}, a, b, x, y)$ with $x \in a$, $y \in b$, from which a and b are recoverable when the set is nonempty, and which is empty iff a or b is empty — in the degenerate empty cases let tr take the atom branch; reflected types $\ulcorner v \urcorner$ are nonempty (they contain $[\![p_v]\!]$) so $\mathrm{tr}([\![\ulcorner v \urcorner]\!]) = v$ for all ground CL terms v, by induction (for the base cases: $[\![\mathsf{K}_0]\!]$ and $[\![\mathsf{S}_0]\!]$ are distinguishable by cardinality). Extend weak reduction to trees with atoms (atoms are inert constants). Interpret

$$[\![R]\!](a, b)(x, y) \;=\; \begin{cases} \{\emptyset\} & \text{if } \mathrm{tr}(a) \to^* \mathrm{tr}(b), \\ \emptyset & \text{otherwise} \end{cases}$$

(element arguments ignored). All six premises are valid: (refl) and (trans) because $\to^*$ is a preorder on trees; (congL), (congR) because $\mathrm{tr}([\![\mathsf{A}]\!](a,b)) = \mathrm{tr}(a) \cdot \mathrm{tr}(b)$ whenever the guard-supplied elements witness nonemptiness of a and b (they do: the premise hypotheses provide $u \in a$ etc.); (redK), (redS) because $K\,x\,y \to x$ and $S\,x\,y\,z \to xz(yz)$ hold for arbitrary trees x, y, z. The goal is not valid: $\mathrm{tr}([\![\ulcorner w \urcorner]\!]) = w \not\to^* w' = \mathrm{tr}([\![\ulcorner w' \urcorner]\!])$.

Claim 2: for every finite monomorphisation Θ of P_{CL}, the relation "$\mathrm{Inst}_\Theta \models_E g(w,w')$" is decidable, uniformly in (Θ, w, w'). Let T_0 be the finite set of closed types occurring in the instances of Θ together with $\ulcorner w \urcorner, \ulcorner w' \urcorner$, closed under syntactic subterms. Emptiness of the denotation of a closed data type is uniform across standard models and decidable (a type $\mathsf{A}\,\sigma\,\rho$ is inhabited iff both arguments are; $\mathsf{K}_0, \mathsf{S}_0$ are inhabited; in general, for our inductive scheme, inhabitation of closed instances is computed by the obvious fixed-point marking). Consider *element-uniform* interpretations of R: those with $[\![R]\!](a,b)(x,y)$ independent of (x,y). Define $r_\Theta \subseteq T_0 \times T_0$ as the least relation closed under the rules read off the instances in Θ: an instance of (trans) at types (σ,ρ,χ) contributes "$(\sigma,\rho) \in r \wedge (\rho,\chi) \in r \Rightarrow (\sigma,\chi) \in r$" provided σ, ρ, χ are inhabited (otherwise the instance is vacuous), and similarly for the other premise schemes; this is a finite monotone system, so r_Θ is computable. We show: $\mathrm{Inst}_\Theta \models_E g(w,w')$ iff $(\ulcorner w \urcorner, \ulcorner w' \urcorner) \in r_\Theta$. If $(\ulcorner w \urcorner, \ulcorner w' \urcorner) \in r_\Theta$: by induction on the closure derivation, in *every* model validating the instances, $[\![R]\!]$ relates the canonical points of the corresponding types (inhabitation supplies the element witnesses needed to apply the guarded instances, and the derivation supplies the antecedents), so the goal is valid. If $(\ulcorner w \urcorner, \ulcorner w' \urcorner) \notin r_\Theta$: take the canonical tagged model with the element-uniform interpretation $[\![R]\!](a,b) = \{\emptyset\}$ iff $(\sigma,\rho) \in r_\Theta$ for some $\sigma, \rho \in T_0$ with $[\![\sigma]\!] = a$, $[\![\rho]\!] = b$ — in the canonical tagged model distinct closed types in T_0 have distinct denotations (distinguishable by tr and cardinality as above), so this is well-defined — and $\emptyset$ otherwise. Every instance in Θ is then valid: its types lie in T_0, and r_Θ is closed under exactly the rule the instance expresses (vacuity matching the emptiness side conditions). The goal fails since $(\ulcorner w \urcorner, \ulcorner w' \urcorner) \notin r_\Theta$.

Conclusion. Suppose an algorithm as in the statement existed. Given ground CL terms w, w', compute $\Theta := M(E_{\mathrm{CL}}, P_{\mathrm{CL}}, g(w,w'))$ and decide $\mathrm{Inst}_\Theta \models g(w,w')$ by Claim 2. If it holds, then (instances being consequences, Lemma 5.3) $P_{\mathrm{CL}} \models g(w,w')$, so $w \to^* w'$ by Claim 1. If it fails, then by the assumed property of M (contrapositive), $P_{\mathrm{CL}} \not\models g(w,w')$, so $w \not\to^* w'$ by Claim 1. This decides $w \to^* w'$ — contradiction. □

Remark 5.15. The theorem justifies the two design decisions of this section: instance selection must be heuristic and bounded (Definition 5.11), and completeness should be recovered, when desired, by the *keep* policy (Proposition 5.7) rather than by cleverer instance selection. This mirrors the design of Bobot–Paskevich's Dis transformation, which keeps the polymorphic original alongside the instances and is complete for exactly that reason [BP11], and Sledgehammer's heuristic monomorphisation, which drops generics and accepts incompleteness [BBPS16, §5.6]. The guard-erasure results of Section 7 apply to the *drop* policy: erasability is a property of fully monomorphic problems.

6 Signature specialisation

After monomorphisation with the *drop* policy, the guard structure of the problem is ground: every T-atom has a closed type encoding as its second argument. The problem is nevertheless still written in the "apply-everything" dialect: a single binary @, a single binary T, a single unary Pr. *Signature specialisation* rewrites it, occurrence by occurrence, into an ordinary first-order signature: unary guard predicates P_τ, n-ary function symbols, n-ary predicate symbols. Two constraints shape the definition. First, the translation is to remain *shallow*: specialisation is a per-occurrence renaming and introduces *no bridging axioms* connecting the

specialised symbols back to T, $@$ or Pr.[2] Second, soundness must be unconditional, while proof-preservation may legitimately be restricted to the fragment where the rewriting is total.

Throughout this section (Φ, γ) is a translated problem (Definition 4.6), possibly after monomorphisation (Definition 5.5).

Definition 6.1 (Specialisation maps)**.** Let $\mathcal{T}$ be a finite set of closed CIC$^-$ types and kind-level terms (in practice: all closed second arguments of T-atoms in $\Phi \cup \{\gamma\}$, plus Prop), and let $\mathcal{H}$ be a finite set of pairs (c, n) of a constant (original or lifted) and an arity. The specialisation $\mathcal{S}$ of a formula is the result of exhaustively applying the following rewrites, innermost first, to its subterms and atoms:

(S1) *Type-argument fusion:* an occurrence $\hat{c} @ \mathsf{C}(\tau_1) @ \cdots @ \mathsf{C}(\tau_j) @ t_1 @ \cdots$ where c's type begins with j binders of domain Type_1 and all $\tau_i \in \mathcal{T}$ are closed, is rewritten to $\hat{c}_{\langle \vec{\tau} \rangle} \, @ \, t_1 \, @ \cdots$, where $\hat{c}_{\langle \vec{\tau} \rangle}$ is a fresh constant indexed by $(c, \vec{\tau})$.

(S2) *Guard specialisation:* an atom $\mathsf{T}(t, u)$ where u is syntactically $\mathsf{C}(\tau)$ (after (S1) on u's arguments, if any) for some $\tau \in \mathcal{T}$, is rewritten to $\mathsf{P}_\tau(t)$, a fresh unary predicate indexed by τ.

(S3) *Arity flattening:* an occurrence $d @ t_1 @ \cdots @ t_n$ with head a constant d (possibly produced by (S1)) and $(d, n) \in \mathcal{H}$, not itself an argument of a further $@$ with the same head chain, is rewritten to $d^{(n)}(t_1, \ldots, t_n)$, a fresh n-ary function symbol; for $n = 0$ the constant is kept as is.

(S4) *Predicate flattening:* an atom $\mathsf{Pr}(d^{(n)}(t_1, \ldots, t_n))$ or $\mathsf{Pr}(d)$ with d a constant is rewritten to $d^{(n)}_{\mathsf{P}}(t_1, \ldots, t_n)$, a fresh n-ary predicate symbol.

$\mathcal{S}(\Phi, \gamma) = (\{\, \mathcal{S}(\varphi) \mid \varphi \in \Phi \,\}, \mathcal{S}(\gamma))$. The rewriting is *total* on (Φ, γ) if in the result the symbols T, Pr and $@$ do not occur at all.

(S3) and (S4) are the classical optimisations of Meng and Paulson [MP08], already performed by CoqHammer [CK18, §5.5]; (S1) is the symbol-discrimination step of monomorphisation in the sense of [BBPS16, §5.6] and Dis of [BP11]; (S2) is the guard-specialisation this paper is after. All four are instances of one soundness principle:

Theorem 6.2 (Soundness of specialisation)**.** *Let (Φ, γ) be $\mathcal{M}$-sound via the expansion $\mathfrak{M}$ of $\mathfrak{M}^*$ (Definition 4.12). Then $\mathcal{S}(\Phi, \gamma)$ is $\mathcal{M}$-sound.*

Proof. Expand $\mathfrak{M}$ to $\mathfrak{M}'$ by interpreting the fresh symbols by the value functions of the patterns they replace:

$$\begin{aligned}
\hat{c}_{\langle \vec{\tau} \rangle} &\mapsto \mathrm{ap}(\cdots \mathrm{ap}([\![c]\!], [\![\tau_1]\!]) \cdots, [\![\tau_j]\!]),\\
\mathsf{P}_\tau &\mapsto \{\, a \in \mathcal{D} \mid a \in [\![\tau]\!] \,\},\\
d^{(n)} &\mapsto (a_1, \ldots, a_n) \mapsto \mathrm{ap}(\cdots \mathrm{ap}(d^{\mathfrak{M}}, a_1) \cdots, a_n),\\
d^{(n)}_{\mathsf{P}} &\mapsto \{\, (a_1, \ldots, a_n) \mid \emptyset \in \mathrm{ap}(\cdots \mathrm{ap}(d^{\mathfrak{M}}, a_1) \cdots, a_n) \,\}.
\end{aligned}$$

By construction, each rewrite step replaces a term (resp. atom) by one with the same value (resp. truth value) in $\mathfrak{M}'$ under every valuation: for (S1) and (S3) this is the definition of the fresh symbol's interpretation; for (S2), $\mathsf{T}^{\mathfrak{M}}(a, [\![\tau]\!])$ iff $a \in [\![\tau]\!]$ iff $\mathsf{P}_\tau^{\mathfrak{M}'}(a)$, using that $\mathsf{C}(\tau)$ evaluates to $[\![\tau]\!]$ (Lemma 4.9, extended to (S1)-images by the first line); for (S4), $\mathsf{Pr}^{\mathfrak{M}}(b)$ iff $\emptyset \in b$. By induction on formulas, $\mathfrak{M}' \models \varphi \leftrightarrow \mathcal{S}(\varphi)$ pointwise for every $\varphi \in \Phi \cup \{\gamma\}$. Hence $\mathfrak{M}' \models \mathcal{S}(\Phi)$, and $\mathfrak{M}' \models \mathcal{S}(\gamma)$ implies $\mathfrak{M} \models \gamma$, which implies validity of the pending goal. □

[2] The CoqHammer implementation currently prints $\mathsf{T}(x, c\, t_1 \cdots t_n)$ as $c_{\$t}(t_1, \ldots, t_n, x)$ *and* emits an unconditional bridging axiom $\forall \vec{x} y.\ c_{\$t}(\vec{x}, y) \leftrightarrow \mathsf{T}(y, c\, \vec{x})$ [CK18, §5.5], which keeps the generic apparatus alive in every problem. The design here eliminates the bridge; the price is a completeness loss quantified by Theorem 6.4: none on the first-order core, only outside it.

Soundness costs nothing; the substantive question is whether specialisation *loses* proofs, given that no bridging axioms tie P_τ to T. On the first-order core it does not:

Lemma 6.3 (Totality on the first-order core)**.** *Suppose every premise of the (drop-policy monomorphised) problem is monomorphic, and every quantified variable in the premises and goal has as its type a closed data type that is not a product, with no quantification over propositions. Before specialising,* prune *the axiom store: remove the universe atom (D4) (needed only to discharge* $\mathsf{T}(\cdot, \ulcorner \mathsf{Type}_1 \urcorner)$ *guards, which do not occur in monomorphic problems) and the application-typing axioms (*Δ*-app), whose bodies contain variable-headed applications* $z \mathbin{@} x$*; these axioms exist to let a* quantified *function variable be applied, and product-typed quantified variables are excluded here, while for constant-headed terms their derivational content is subsumed by the* Flat *axioms. Pruning removes premises, so it trivially preserves* $\mathcal{M}$*-soundness. Take* $\mathcal{T}$ *to be all closed second arguments of surviving* T*-atoms (including membership atoms at product types, whose encodings are closed) and* $\mathcal{H}$ *all (head, arity) pairs of constant-headed application occurrences. Then* $\mathcal{S}$ *is total on the pruned translated problem.*

Proof. We check that no occurrence of T, Pr, $@$ survives. T-atoms: guards of quantified variables have closed non-product data types, whose encodings are closed terms in $\mathcal{T}$, so (S2) applies; the surviving typing atoms and the Flat hypotheses and conclusions have closed non-product type encodings (monomorphic declarations after instantiating the implicit premises, Definition 5.5), so (S2) applies. $@$: after pruning, every application occurrence has a constant head — variable heads arise only from variables of product or Prop type, excluded in the premises and goal, and from the (Δ-app) axioms, which are pruned — so (S1)/(S3) apply exhaustively. Pr: after (S3), every Pr-argument is a constant-headed flattened term, so (S4) applies. □

Theorem 6.4 (Faithfulness of total specialisation)**.** *If* $\mathcal{S}$ *is total on* (Φ, γ) *and* Φ *contains no* $\approx$*-equations between type-level terms (guaranteed for translated problems by Lemma 4.7), then*

$$\Phi \vdash_{\mathrm{c}} \gamma \qquad \Longleftrightarrow \qquad \mathcal{S}(\Phi) \vdash_{\mathrm{c}} \mathcal{S}(\gamma).$$

Proof. By completeness it suffices to transfer models in both directions.

($\Leftarrow$ direction of provability, i.e. model transfer from Φ-side to $\mathcal{S}$-side) Let $\mathfrak{M} \models \Phi \cup \{\neg\gamma\}$. Interpret the fresh symbols over the same domain by the value functions of their patterns, exactly as in the proof of Theorem 6.2 but with $\mathfrak{M}$ in place of $\mathfrak{M}^*$'s expansion. The same pointwise equivalence gives a model of $\mathcal{S}(\Phi) \cup \{\neg\mathcal{S}(\gamma)\}$.

($\Rightarrow$ direction, model transfer from $\mathcal{S}$-side) Let $\mathfrak{M}' \models \mathcal{S}(\Phi) \cup \{\neg\mathcal{S}(\gamma)\}$ with domain $\mathcal{D}'$. By totality, the symbols T, Pr, $@$, the raw constants with nonzero occurring arity, and the type-encoding constants do not occur in the specialised problem, so we may interpret them freely; 0-ary raw constants that survive specialisation unchanged keep their $\mathfrak{M}'$-interpretation. Build $\mathfrak{M}$ with domain

$$\mathcal{D} \;:=\; \mathcal{D}' \;\uplus\; \{\, \mathbf{t}_\tau \mid \tau \in \mathcal{T}^{\downarrow} \,\} \;\uplus\; \{\, \mathbf{p}_{d,\vec{a}} \mid (d,n) \in \mathcal{H},\ \vec{a} \in (\mathcal{D}')^{<n} \,\}$$

where $\mathcal{T}^{\downarrow}$ is the subterm closure of $\mathcal{T}$ (finitely many fresh points $\mathbf{t}_\tau$, one per closed type-level term, pairwise distinct and distinct from $\mathcal{D}'$), and the $\mathbf{p}$-points represent partial applications. Interpret: each type-encoding constant and each application step among type encodings so that the value of $\mathsf{C}(\tau)$ is $\mathbf{t}_\tau$ (possible since the $\mathbf{t}$-points are free and the syntax is finite: interpret $\hat{I}$ as $\mathbf{t}_I$, $\mathrm{ap}(\mathbf{t}_{I\,\vec{\sigma}<}, \mathbf{t}_\sigma) = \mathbf{t}_{I\,\vec{\sigma}<\sigma}$, etc., with an arbitrary default elsewhere); ap on a raw constant head d and arguments $\vec{a} \in \mathcal{D}'$ climbs the $\mathbf{p}$-points and, at the n-th argument for $(d,n) \in \mathcal{H}$, returns $(d^{(n)})^{\mathfrak{M}'}(\vec{a})$ if $d^{(n)}$ occurs in the specialised problem, and otherwise a fresh

terminal **p**-point; fused heads $\hat{c}_{\langle\vec{\tau}\rangle}$ are reached as the value of $\hat{c}$ applied to $\mathbf{t}_{\tau_1}, \ldots$, again via dedicated **p**-points;

$$\begin{aligned} \mathsf{T}^{\mathfrak{M}}(a,b) &\iff b = \mathbf{t}_\tau \text{ for some } \tau \in \mathcal{T} \text{ and } a \in \mathsf{P}_\tau^{\mathfrak{M}'}, \\ \mathsf{Pr}^{\mathfrak{M}}(a) &\iff a = \text{terminal point of } (d, \vec{a}) \text{ and } (d_{\mathsf{P}}^{(n)})^{\mathfrak{M}'}(\vec{a}), \end{aligned}$$

with Pr false elsewhere and $\mathsf{P}_\tau^{\mathfrak{M}'} \subseteq \mathcal{D}'$ read as a subset of $\mathcal{D}$.

We claim $\mathfrak{M} \models \varphi \iff \mathfrak{M}' \models \mathcal{S}(\varphi)$ for every $\varphi \in \Phi \cup \{\gamma\}$ and, more generally, for every subformula under valuations ν sending each free variable into $\mathcal{D}'$. The proof is by induction on φ, with an auxiliary term-level statement: for every term t occurring in φ that is (i) an argument of $\approx$, of a specialised guard atom, or of a flattened symbol, the $\mathfrak{M}$-value of t under ν equals the $\mathfrak{M}'$-value of $\mathcal{S}(t)$ under ν. For the term statement, note that such t is a fully applied constant-headed chain or a variable (by totality), and the interpretation of ap was constructed to make the chain land on exactly $(d^{(n)})^{\mathfrak{M}'}(\vec{a})$; variables are shared. Atoms: $\approx$-atoms compare $\mathcal{D}'$-values equal in both structures — here the absence of type-level equations matters: both sides are data-level chains landing in $\mathcal{D}'$, never **t**- or **p**-points; guard atoms $\mathsf{T}(t, \mathsf{C}(\tau))$ hold in $\mathfrak{M}$ iff $\mathsf{P}_\tau^{\mathfrak{M}'}$ holds of the (shared) value of t, which is the truth condition of the specialised atom; Pr-atoms likewise via the terminal-point clause. Quantifiers: every quantifier of φ is guarded (inspection of Definition 4.5: (F2), (F5), the guarded closures of all axiom schemes) by a guard whose $\mathfrak{M}$-extension is $\mathsf{P}_\tau^{\mathfrak{M}'} \subseteq \mathcal{D}'$; hence the effective range of every quantifier in $\mathfrak{M}$ is inside $\mathcal{D}'$, the valuations stay in $\mathcal{D}'$, and the induction hypothesis applies. (Universally quantified formulas hold vacuously at the fresh points, which fail every guard; existentials find their witnesses in $\mathcal{D}'$ on both sides.)

Thus $\mathfrak{M} \models \Phi \cup \{\neg\gamma\}$, completing the transfer. □

Remark 6.5 (Outside the first-order core). When higher-order quantification remains (e.g. a monomorphic premise $\forall f{:}\mathsf{nat} \to \mathsf{nat}. \ldots$), occurrences $f \mathbin{@} x$ with variable head survive specialisation, @ stays in the signature, and Theorem 6.4 does not apply — specialisation remains sound (Theorem 6.2) but may lose proofs that exploited the identity of $\mathsf{T}(\cdot, \ulcorner \mathsf{nat} \to \mathsf{nat} \urcorner)$ with the guard of some other occurrence. The bridge axioms of the implementation buy back exactly these proofs at the cost of reintroducing the generic apparatus; under the shallow-translation constraint we accept the loss and note that on hammer benchmarks the essentially first-order goals dominate [CK18].

Remark 6.6 (Open guard shapes). (S2) requires the guard's type argument to be closed. A guard $\mathsf{T}(y, \mathsf{C}(\mathsf{list}\,(Q\,x)))$ with a free data variable x (arising under a binder) can be specialised too, by a *shape* predicate: rewrite to $\mathsf{P}_{\mathsf{list}\,(Q\,\cdot)}(x, y)$, a fresh binary predicate indexed by the type pattern with its free variables abstracted. Theorem 6.2 extends verbatim — interpret the shape predicate by the value function $(a, b) \mapsto (b \in [\![\mathsf{list}\,(Q\,x)]\!]_{[x \mapsto a]})$ — so shape specialisation is sound; we do not claim faithfulness for it. This settles, affirmatively for soundness, the question raised in the CoqHammer development notes of whether guards over open types may be compressed into parameterised predicates.

7 Monotonicity-based guard erasure

Guards are the tax the untyped encoding pays for soundness. This section determines exactly when the tax can be waived: for which types τ the guard atoms $\mathsf{P}_\tau(\cdot)$ may be deleted from a fully monomorphised, fully specialised problem without compromising the soundness invariant. The answer — τ must be *witnessed inhabited* and *monotonic*, with monotonicity established either by the absence of naked variables or by a provable-infinity criterion that the translation's own injectivity and discrimination axioms supply for recursive inductive types

— transplants the monotonicity framework of Claessen, Lilliestrom and Smallbone [CLS11] and Blanchette, Böhme, Popescu and Smallbone [BBPS16] from many-sorted first-order logic to the guarded unsorted problems generated from CIC$^-$. The transplant is not routine: in the guarded world all types share one domain, an individual may inhabit several types, and the empty type is expressible. Both phenomena show up as necessary side conditions (Propositions 7.18 and 7.19).

Throughout this section, formulas are in negation normal form (classically without loss of generality), and (Φ, γ) is a totally specialised problem (Definition 6.1 and Lemma 6.3); we write $\mathcal{G} = \{\, \mathsf{P}_\tau \mid \tau \in \mathcal{T} \,\}$ for its finite family of guard predicates and $R \coloneqq \Phi \cup \{\mathrm{nnf}(\neg\gamma)\}$ for the refutation set.

7.1 Erasure, and what must be proved

Definition 7.1 (Erasure)**.** For $\Sigma_0 \subseteq \mathcal{T}$, the erasure $\mathcal{E}_{\Sigma_0}(\varphi)$ replaces every atom $\mathsf{P}_\tau(t)$ with $\tau \in \Sigma_0$ by $\top$ (in NNF: positive occurrences by $\top$, negative occurrences $\neg\mathsf{P}_\tau(t)$ by $\bot$), followed by the evident Boolean simplifications. In particular guarded quantifiers lose their guards: $\forall x.\, (\neg\mathsf{P}_\tau(x) \vee \varphi) \mapsto \forall x.\, \varphi$ and $\exists x.\, (\mathsf{P}_\tau(x) \wedge \varphi) \mapsto \exists x.\, \varphi$, and typing-axiom conclusions $\mathsf{P}_\tau(f(\vec{x}))$ become trivial. $\mathcal{E}_{\Sigma_0}$ is applied to every element of R.

Lemma 7.2 (Erasure at total guards)**.** *If $\mathfrak{M}$ is a structure in which $\mathsf{P}_\tau^{\mathfrak{M}}$ is the whole domain for every $\tau \in \Sigma_0$, then $\mathfrak{M} \models \varphi$ iff $\mathfrak{M} \models \mathcal{E}_{\Sigma_0}(\varphi)$, for every formula φ.*

Proof. Substituting $\top$ for atoms that are true under every valuation does not change the truth value of any formula; by induction on φ. □

Proposition 7.3 (Erasure never loses proofs)**.** *If $\Phi \vdash_{\mathrm{c}} \gamma$ then $\mathcal{E}_{\Sigma_0}(\Phi) \vdash_{\mathrm{c}} \mathcal{E}_{\Sigma_0}(\gamma)$, for any Σ_0.*

Proof. Contrapositively, let $\mathfrak{M} \models \mathcal{E}_{\Sigma_0}(R)$. Let $\mathfrak{M}'$ agree with $\mathfrak{M}$ except that $\mathsf{P}_\tau^{\mathfrak{M}'}$ is the whole domain for each $\tau \in \Sigma_0$. The erased formulas do not mention the reinterpreted predicates, so $\mathfrak{M}' \models \mathcal{E}_{\Sigma_0}(R)$, and by Lemma 7.2 $\mathfrak{M}' \models R$. □

So erasure is dangerous in one direction only: it may *prove more*. The soundness obligation is:

Definition 7.4 (Inflatability)**.** R is Σ_0-*inflatable* if, whenever R is satisfiable, R has a model in which P_τ is total (the whole domain) for every $\tau \in \Sigma_0$.

Theorem 7.5 (Erasure soundness from inflatability)**.** *Let (Φ, γ) be $\mathcal{M}$-sound and suppose $R = \Phi \cup \{\mathrm{nnf}(\neg\gamma)\}$ is Σ_0-inflatable. If $\mathcal{E}_{\Sigma_0}(\Phi) \vdash_{\mathrm{c}} \mathcal{E}_{\Sigma_0}(\gamma)$, then the pending goal is valid in $\mathcal{M}$.*

Proof. Suppose the goal is not valid. By $\mathcal{M}$-soundness there is an expansion $\mathfrak{M}$ of $\mathfrak{M}^*$ with $\mathfrak{M} \models \Phi$, and by the contrapositive of Definition 4.12(ii), $\mathfrak{M} \not\models \gamma$; hence R is satisfiable. By inflatability, R has a model with all Σ_0-guards total, which by Lemma 7.2 is a model of $\mathcal{E}_{\Sigma_0}(R) = \mathcal{E}_{\Sigma_0}(\Phi) \cup \{\mathrm{nnf}(\neg\mathcal{E}_{\Sigma_0}(\gamma))\}$ — contradicting the assumed provability. □

The rest of the section establishes checkable sufficient conditions for inflatability, by passing through many-sorted logic.

7.2 Sorted cores and relativisation

Definition 7.6 (Sorted core)**.** A totally specialised problem R over guard family $\mathcal{G}$ (indexed by $\mathcal{T}$, now regarded as a set of *sorts*) is a *sorted core* if it splits as $R = R_{\log} \uplus R_{\mathrm{ty}}$ such that:

(SC1) every quantifier in $R_{\log}$ and in the guarded prefixes of R_{ty} is guarded: $\forall x.\, (\neg\mathsf{P}_\tau(x) \vee \cdots)$ or $\exists x.\, (\mathsf{P}_\tau(x) \wedge \cdots)$; the sort of the bound variable is the guard's index;

(SC2) there is a *sort assignment*: each function symbol f carries a signature $(\vec{\sigma};\varsigma)$ over $\mathcal{T}$, each predicate symbol other than the guards a signature $(\vec{\sigma})$, such that every occurrence of every symbol in R is sort-correct (arguments of the declared sorts, computed from variable sorts upward), and every $\approx$-literal compares terms of equal sort;

(SC3) R_{ty} consists of, for each function symbol f with signature $(\vec{\sigma};\varsigma)$, the *typing axiom* $\forall\vec{x}.\,\big(\bigvee_i \neg\mathsf{P}_{\sigma_i}(x_i)\big) \vee \mathsf{P}_\varsigma(f(\vec{x}))$ (for 0-ary f: the ground *typing fact* $\mathsf{P}_\varsigma(f)$); and guard atoms occur in R_{log} only in the guard positions of (SC1);

(SC4) every sort $\tau \in \mathcal{T}$ is *witnessed*: there is a ground term w_τ that is sort-correct of sort τ.

The *sorted companion* R^{ms} is the many-sorted problem over sorts $\mathcal{T}$ (with the signatures of (SC2)) whose formulas are those of R_{log} with each guarded quantifier replaced by the corresponding sorted quantifier ($\forall x{:}\tau$, $\exists x{:}\tau$) and guard literals deleted; R_{ty} has no counterpart (it is absorbed into the sorted signature discipline). Many-sorted semantics is standard: nonempty domains $(D_\tau)_{\tau\in\mathcal{T}}$, sort-respecting interpretations, equality per sort.

Lemma 7.7 (Pipeline problems are sorted cores)**.** *The totally specialised translation of a drop-policy monomorphised first-order-core problem (Lemma 6.3), restricted to premises whose quantified types are witnessed, is a sorted core.*

Proof. (SC1): all quantifiers of the translation are guarded ((F2), (F5), and the guarded closures of every axiom scheme in Definitions 4.5 and 4.6), and (S2) turns each guard into a P_τ-atom. (SC2): sorts are read off the CIC$^-$ types: each flattened symbol $d^{(n)}$ or fused constant descends from a constant or lifted constant with a closed CIC$^-$ type, which after monomorphisation is a closed iterated product over non-product data types; its signature is the list of argument types and result type. Sort-correctness of occurrences follows from well-typedness of the CIC$^-$ terms being translated (Lemma 4.9 respects types by (M1)), and $\approx$-literals compare translations of CIC$^-$ terms of one type ((F4), levelled equalities; Lemma 4.7 excludes type-level equations). (SC3): the specialised Flat axioms and typing atoms have exactly the displayed shapes. (SC4) is the restriction in the statement. □

Lemma 7.8 (Witness truth)**.** *In every model $\mathfrak{M}$ of a sorted core R, and for every $\tau \in \mathcal{T}$: $\mathsf{P}_\tau^{\mathfrak{M}} \neq \emptyset$; indeed the value of any ground sort-correct term of sort τ lies in $\mathsf{P}_\tau^{\mathfrak{M}}$.*

Proof. By induction on the ground term, using the typing facts for constants and the typing axioms for compound terms (their guard premises hold by the induction hypothesis). □

Theorem 7.9 (Relativisation)**.** *A sorted core R is satisfiable iff its sorted companion R^{ms} is satisfiable. Moreover, from a model of R one obtains a model of R^{ms} with domains $D_\tau = \mathsf{P}_\tau^{\mathfrak{M}}$, and conversely from a model of R^{ms} a model of R whose guard extensions are (disjoint copies of) the sorted domains.*

Proof. ($\Rightarrow$) Let $\mathfrak{M} \models R$ with domain $\mathcal{D}$. Set $D_\tau := \mathsf{P}_\tau^{\mathfrak{M}}$, nonempty by Lemma 7.8. For f with signature $(\vec{\sigma};\varsigma)$ define f^{ms} as the restriction of $f^{\mathfrak{M}}$ to $\prod_i D_{\sigma_i}$; the typing axiom of f (true in $\mathfrak{M}$) guarantees values in D_ς, so this is a legal sorted interpretation. Restrict predicates likewise. We claim: for every subformula φ of a formula of R_{log} and every sorted valuation ξ (which is also an $\mathfrak{M}$-valuation), $\mathfrak{M}, \xi \models \varphi$ iff $\mathcal{N}, \xi \models \varphi^{\mathrm{ms}}$, where $\mathcal{N}$ is the sorted structure just defined. Induction on φ: atoms and their negations are interpreted by restrictions, and all argument values lie in the right domains by (SC2) and an inner induction on terms; the connective cases are trivial; a guarded $\forall x.\,(\neg\mathsf{P}_\tau(x) \vee \psi)$ holds in $\mathfrak{M}$ iff ψ holds for all x-values in $\mathsf{P}_\tau^{\mathfrak{M}} = D_\tau$, which by the induction hypothesis is the truth condition of $\forall x{:}\tau.\,\psi^{\mathrm{ms}}$; dually for $\exists$. Hence $\mathcal{N} \models R^{\mathrm{ms}}$.

($\Leftarrow$) Let $\mathcal{N} \models R^{\mathrm{ms}}$ with domains (D_τ). Let $\mathcal{D} := \biguplus_\tau D_\tau$ (tagged disjoint union; write $\iota_\tau : D_\tau \to \mathcal{D}$ for the injections). Interpret $\mathsf{P}_\tau^{\mathfrak{M}} := \iota_\tau(D_\tau)$; for f with signature $(\vec{\sigma}; \varsigma)$ let $f^{\mathfrak{M}}(\vec{u}) := \iota_\varsigma(f^{\mathcal{N}}(\vec{d}))$ when $u_i = \iota_{\sigma_i}(d_i)$ for all i, and an arbitrary fixed element of $\mathcal{D}$ otherwise; predicates: true on sort-matching tuples exactly when true in $\mathcal{N}$, false otherwise. The typing axioms hold by construction (guard-passing arguments are sort-matching). For $R_{\log}$, the same induction as above, run in reverse, shows each formula true in $\mathfrak{M}$: guarded quantifiers range over $\iota_\tau(D_\tau)$, which is a bijective copy of D_τ; sort-correct terms under sort-conforming valuations never take the default branch; $\approx$-literals compare values within one ι_τ-image, where ι_τ is injective. □

We also record the standard corollary used below.

Corollary 7.10 (Countable models, many-sorted)**.** *If a sorted problem T over a countable signature and finitely many sorts (all interpreted nonempty) is satisfiable, it has a model with all domains countable.*

Proof. Encode T as a guarded unsorted problem by reversing the construction of Theorem 7.9 (guard each quantifier, add typing axioms and one witness constant per sort — this is the classical relativisation, and the two directions proved above did not use any property of R beyond the sorted-core shape, so they apply). The unsorted problem is satisfiable, hence by the Löwenheim–Skolem theorem for countable signatures [CK90] it has a countable model; transferring back through Theorem 7.9($\Rightarrow$) yields a sorted model whose domains are subsets of a countable set, and they are nonempty by Lemma 7.8. □

7.3 Monotonicity

Definition 7.11 (Monotonic sort [BBPS16, Def. 5.1])**.** Let T be a many-sorted problem and S a set of its sorts. S is *monotonic* in T if for every model $\mathcal{N}$ of T there is a model $\mathcal{N}'$ of T such that D'_τ is infinite for every $\tau \in S$ and $D'_\rho = D_\rho$ for $\rho \notin S$.

Definition 7.12 (Naked variables [BBPS16, Def. 5.6])**.** For NNF formulas: $\mathrm{NV}(p(\vec{t})) = \mathrm{NV}(\neg p(\vec{t})) = \mathrm{NV}(t_1 \not\approx t_2) = \emptyset$; $\mathrm{NV}(t_1 \approx t_2) = \{t_1, t_2\} \cap \mathcal{V}$ (the variables among the two sides); $\mathrm{NV}(\varphi \wedge \psi) = \mathrm{NV}(\varphi \vee \psi) = \mathrm{NV}(\varphi) \cup \mathrm{NV}(\psi)$; $\mathrm{NV}(\forall x{:}\tau.\, \varphi) = \mathrm{NV}(\varphi)$; $\mathrm{NV}(\exists x{:}\tau.\, \varphi) = \mathrm{NV}(\varphi) \setminus \{x\}$. For a problem, $\mathrm{NV}(T) = \bigcup_{\varphi \in T} \mathrm{NV}(\varphi)$. A sort τ is *NV-safe* in T if no variable of sort τ belongs to $\mathrm{NV}(T)$.

Lemma 7.13 (Cloning; adapted from [BBPS16, Thm. 5.9] and [CLS11, §2.3])**.** *Let τ be NV-safe in the sorted problem T and let $\mathcal{N} \models T$. For any infinite cardinal $\kappa \geq |D_\tau|$ there is $\mathcal{N}' \models T$ with $|D'_\tau| = \kappa$ and $D'_\rho = D_\rho$ for all $\rho \neq \tau$.*

Proof. Let E be a set of fresh elements with $|D_\tau \uplus E| = \kappa$ and put $D'_\tau := D_\tau \uplus E$, $D'_\rho := D_\rho$ otherwise. Fix $e_0 \in D_\tau$ (nonempty) and define the retraction $r : D'_\tau \to D_\tau$ by $r(e) = e_0$ for $e \in E$ and $r(d) = d$ for $d \in D_\tau$; extend r to other sorts as the identity. Interpret every function and predicate symbol by precomposition with r: $f^{\mathcal{N}'}(\vec{d}) := f^{\mathcal{N}}(r(\vec{d}))$, $p^{\mathcal{N}'}(\vec{d}) :\iff p^{\mathcal{N}}(r(\vec{d}))$. For a valuation ξ' over $\mathcal{N}'$ let $\xi := r \circ \xi'$.

Fact 1. For every term t that is not a variable of sort τ: $\mathrm{val}_{\mathcal{N}',\xi'}(t) = \mathrm{val}_{\mathcal{N},\xi}(t)$; and for a variable x of sort τ: $r(\mathrm{val}_{\mathcal{N}',\xi'}(x)) = \mathrm{val}_{\mathcal{N},\xi}(x)$. (Induction on t; function values are computed through r, so they land in the original domains and agree.)

Fact 2. For every NNF formula φ and every ξ' such that $\xi'(x) \in D_\tau$ (no clone) for each free $x \in \mathrm{NV}(\varphi)$ of sort τ: if $\mathcal{N}, \xi \models \varphi$ then $\mathcal{N}', \xi' \models \varphi$. Induction on φ:

- Non-equality literals: all argument values retract correctly (Fact 1 and $r \circ \xi' = \xi$), and the interpretations factor through r, so the truth values agree exactly.

- $t_1 \approx t_2$ (positive): each side is either a non-variable or a variable of another sort (then Fact 1 gives equality of values with the $\mathcal{N}$-side) or a naked variable of sort τ — which by the valuation condition is mapped to a non-clone, so again its value coincides with its $\mathcal{N}$-value. Hence the equation transfers.
- $t_1 \not\approx t_2$: let v'_i be the $\mathcal{N}'$-values and v_i the $\mathcal{N}$-values; by Fact 1, $r(v'_i) = v_i$ (for non-variables and other-sort variables even $v'_i = v_i$). From $v_1 \neq v_2$ and $r(v'_1) = v_1 \neq v_2 = r(v'_2)$ we get $v'_1 \neq v'_2$ since r is a function.
- $\wedge, \vee$: immediate.
- $\forall x{:}\sigma.\, \psi$: for arbitrary $d \in D'_\sigma$, the retracted valuation is $\xi[x \mapsto r(d)]$ with $r(d) \in D_\sigma$, so $\mathcal{N}, \xi[x \mapsto r(d)] \models \psi$; the valuation condition for ψ under $\xi'[x \mapsto d]$ holds: if $x \in \mathrm{NV}(\psi)$ had sort τ then $x \in \mathrm{NV}(\forall x.\psi) = \mathrm{NV}(\psi)$ would make τ NV-unsafe in T, contradiction — so either x's sort is not τ or $x \notin \mathrm{NV}(\psi)$, and other free naked variables keep their values. Apply the induction hypothesis.
- $\exists x{:}\sigma.\, \psi$: take the $\mathcal{N}$-witness $d \in D_\sigma \subseteq D'_\sigma$; then $\xi'[x \mapsto d]$ maps x to a non-clone, so the valuation condition holds for ψ even if $x \in \mathrm{NV}(\psi)$, and $r(d) = d$ keeps the retracted valuation aligned. Apply the induction hypothesis.

Since sentences have no free variables, Fact 2 gives $\mathcal{N}' \models T$. □

Lemma 7.14 (Forced infinity is monotonic)**.** *If every model of T interprets τ by an infinite domain, then $\{\tau\}$ is monotonic in T (take $\mathcal{N}' = \mathcal{N}$).* □

Lemma 7.15 (Inductive infinity)**.** *Let R be a sorted core arising from the pipeline (Lemma 7.7) and let τ be the sort of a monomorphic instance $I\,\vec{\sigma}$ of a data inductive type. Suppose R contains, for some constructor c_j of I with a recursive argument position l_0:*

(i) the specialised instance at $\vec{\sigma}$ of c_j's injectivity axiom;

(ii) witnesses (SC4) for the sorts of all non-recursive argument positions of c_j at $\vec{\sigma}$, and for τ itself a witness w_τ whose head constructor is some c_i with $i \neq j$, together with the specialised instance at $\vec{\sigma}$ of the discrimination axiom for the pair (c_i, c_j).

Then D_τ is infinite in every model of the sorted companion R^{ms}.

Proof. Fix a model; let $\vec{w}$ be the values of the non-recursive witnesses and define $h : D_\tau \to D_\tau$ by $h(a) := c_j^{\mathrm{ms}}(\vec{w}, a)$ (a in position l_0). By the injectivity axiom (instantiated at the fixed $\vec{w}$), h is injective. By the discrimination axiom, the value of w_τ (a c_i-headed ground term) is not in the image of c_j^{ms}, in particular not in the image of h. A set admitting an injective, non-surjective self-map is Dedekind-infinite. □

Whether the hypotheses of Lemma 7.15 hold is a simple syntactic check on the environment and the instance set: I has a constructor with a recursive argument whose non-recursive argument types (at $\vec{\sigma}$) have closed inhabitants — e.g. nat; list σ for witnessed σ; trees — while unit, bool, enumerations, and list of an empty type all fail it, as they must. (If every closed inhabitant of τ had head c_j, an innermost such inhabitant would need a smaller closed inhabitant at the recursive position, so a witnessed type with a recursive constructor always has a second, non-recursive constructor to discriminate against; condition (ii) is thus no real restriction beyond witnessing.) Note that the criterion is *internal*: it demands that the injectivity and discrimination axioms be *present in the problem* — which the translation guarantees for inductive types occurring in it — and therefore avoids the soundness gap of infinity-by-introspection reported for Sledgehammer, where a type known infinite in the background theory but not in the problem makes the encoding unsound relative to the problem alone [BBPS16, §7.4].

7.4 The erasure theorem

Definition 7.16 (Erasable types)**.** Let R be a sorted core. A sort τ is *erasable* if it is witnessed (SC4) and either (a) NV-safe in R^{ms} (Definition 7.12), or (b) forced infinite in every model of R^{ms} by Lemma 7.15 (or any other argument). Write Σ_0 for any chosen set of erasable sorts.

Theorem 7.17 (Guard-erasure soundness)**.** *Let $R = \Phi \cup \{\mathrm{nnf}(\neg\gamma)\}$ be a sorted core and Σ_0 a set of erasable sorts. Then R is Σ_0-inflatable. Consequently (Theorem 7.5), if (Φ, γ) is $\mathcal{M}$-sound and $\mathcal{E}_{\Sigma_0}(\Phi) \vdash_{\mathrm{c}} \mathcal{E}_{\Sigma_0}(\gamma)$, the goal is valid in $\mathcal{M}$; and by Proposition 7.3 no first-order proof is lost.*

Proof. Assume R satisfiable; we construct a model of R with all Σ_0-guards total.

Step 1 (sorted, countable). By Theorem 7.9($\Rightarrow$), R^{ms} is satisfiable; by Corollary 7.10 it has a model $\mathcal{N}_1$ with all domains countable (nonempty).

Step 2 (inflate the erasable sorts). Enumerate $\Sigma_0 = \{\tau_1, \ldots, \tau_m\}$ (finite). Define $\mathcal{N}_2$ by treating each τ_i in turn: if τ_i is forced infinite, its domain is already countably infinite and we do nothing; otherwise τ_i is NV-safe, and we apply Lemma 7.13 with $\kappa = \aleph_0$, which changes only D_{τ_i}. NV-safety is a property of the problem R^{ms}, not of the model, so successive applications remain justified; forced-infinite sorts keep their (unchanged, infinite) domains. After the m steps, every $\tau \in \Sigma_0$ has $|D_\tau| = \aleph_0$ and all other domains are unchanged, countable and nonempty.

Step 3 (merge). Let U be a fixed countably infinite set. Choose bijections $b_\tau : D_\tau \to U$ for $\tau \in \Sigma_0$ and injections $j_\rho : D_\rho \hookrightarrow U$ for $\rho \in \mathcal{T} \setminus \Sigma_0$ (possible: the domains are countable). Write emb_σ for b_σ or j_σ as appropriate, and proj_σ for the (partial) inverse, total for $\sigma \in \Sigma_0$. Define the unsorted structure $\mathfrak{M}$ with domain U:

- $\mathsf{P}_\rho^{\mathfrak{M}} := j_\rho(D_\rho)$ for $\rho \notin \Sigma_0$; $\mathsf{P}_\tau^{\mathfrak{M}} := U$ for $\tau \in \Sigma_0$;
- for f with signature $(\vec{\sigma}; \varsigma)$: $f^{\mathfrak{M}}(\vec{u}) := \mathrm{emb}_\varsigma\big(f^{\mathcal{N}_2}(\mathrm{proj}_{\sigma_1}(u_1), \ldots)\big)$ when every $\mathrm{proj}_{\sigma_i}(u_i)$ is defined, and $\mathrm{emb}_\varsigma(d_\varsigma^0)$ (a fixed default $d_\varsigma^0 \in D_\varsigma$) otherwise;
- for a predicate p with signature $(\vec{\sigma})$: $p^{\mathfrak{M}}(\vec{u})$ iff every $\mathrm{proj}_{\sigma_i}(u_i)$ is defined and $p^{\mathcal{N}_2}$ holds of the projections.

Note $\mathrm{proj}_\sigma \circ \mathrm{emb}_\sigma = \mathrm{id}$ for every sort, and proj_τ is total and bijective for $\tau \in \Sigma_0$ — this totality is exactly what Step 2 bought.

Claim: $\mathfrak{M} \models R$, with all Σ_0-guards total. Totality of the Σ_0-guards holds by definition. For the formulas, we prove: for every formula $\varphi \in R_{\log}$, every subformula ψ of φ, and every sort-conforming pair of valuations (ξ over $\mathcal{N}_2$, ν over $\mathfrak{M}$) with $\nu(x) = \mathrm{emb}_{\mathrm{sort}(x)}(\xi(x))$ for the free variables:

$$\mathcal{N}_2, \xi \models \psi^{\mathrm{ms}} \quad \Longleftrightarrow \quad \mathfrak{M}, \nu \models \psi.$$

Terms first: by induction, the $\mathfrak{M}$-value of a sort-ς term t under ν is emb_ς of its $\mathcal{N}_2$-value under ξ (variables by conformity; applications because $\mathrm{proj} \circ \mathrm{emb} = \mathrm{id}$ keeps every projection defined, so the default branch is never taken). Atoms: $p(\vec{t})$ transfers by the same identity; $t_1 \approx t_2$ at sort ς transfers because emb_ς is injective. Connectives: trivial. Quantifier at a sort $\rho \notin \Sigma_0$ (guard kept): the $\mathfrak{M}$-side quantifier is guarded by $\mathsf{P}_\rho^{\mathfrak{M}} = j_\rho(D_\rho)$, a bijective copy of the $\mathcal{N}_2$-range — both sides range over matching values and the induction hypothesis applies. Quantifier at a sort $\tau \in \Sigma_0$ (guard *present* in R; recall we are proving $\mathfrak{M} \models R$, not its erasure): the guard is total, so the $\mathfrak{M}$-side ranges over all of $U = b_\tau(D_\tau)$ — again a bijective copy of the sorted range, and the induction hypothesis applies. This proves every formula of $R_{\log}$ true in $\mathfrak{M}$ (its $^{\mathrm{ms}}$-image being true in $\mathcal{N}_2$).

R_{ty}: for a typing axiom of $f : (\vec{\sigma}; \varsigma)$, let $\vec{u}$ pass the (remaining) guards. For $\sigma_i \in \Sigma_0$ the projection is total; for $\sigma_i \notin \Sigma_0$ the guard $\mathsf{P}_{\sigma_i}(u_i)$ asserts $u_i \in j_{\sigma_i}(D_{\sigma_i})$, so the projection is

defined. Hence $f^{\mathfrak{M}}(\vec{u})$ is emb_ς of a value in D_ς, and the conclusion $\mathsf{P}_\varsigma(f(\vec{u}))$ holds ($= U$ if $\varsigma \in \Sigma_0$, $= j_\varsigma(D_\varsigma) \ni$ the value otherwise). Typing facts and witness terms: likewise.

So $\mathfrak{M} \models R$ with total Σ_0-guards: R is Σ_0-inflatable. □

7.5 Necessity of the side conditions

Proposition 7.18 (Witnessing is necessary)**.** *There is an $\mathcal{M}$-sound sorted-core problem (Φ, γ) with γ's pending goal* invalid *in $\mathcal{M}$, and a* non-witnessed *but NV-safe type τ, such that $\mathcal{E}_{\{\tau\}}(\Phi) \vdash_{\mathrm{c}} \mathcal{E}_{\{\tau\}}(\gamma)$.*

Proof. Let empty be the data inductive type with no constructors, and take the single premise $\varphi = \Pi x{:}\mathsf{empty}.\ \mathsf{O} \doteq_{\mathsf{nat}} \mathsf{S}\,\mathsf{O}$ — a *valid* proposition ($[\![\mathsf{empty}]\!] = \emptyset$ by (M9), so the product is vacuously $\{\emptyset\}$; it is also provable by empty-elimination in CIC). Its specialised translation is $\forall x.\ \neg\mathsf{P}_{\mathsf{empty}}(x) \vee (\hat{\mathsf{O}} \approx \hat{\mathsf{S}}(\hat{\mathsf{O}}))$. The type empty is NV-safe (x occurs in no equation; the inversion axiom for a constructorless type is $\forall x.\ \neg\mathsf{P}_{\mathsf{empty}}(x) \vee \bot$, equation-free), but not witnessed. Take the goal γ translating $\varphi_G = (\mathsf{O} \doteq \mathsf{S}\,\mathsf{O})$, invalid in $\mathcal{M}$ by (M9)(c). Erasing the empty-guard turns the premise into $\forall x.\ \hat{\mathsf{O}} \approx \hat{\mathsf{S}}(\hat{\mathsf{O}})$, from which $\mathcal{E}(\gamma) = \gamma$ follows immediately. □

Proposition 7.19 (Monotonicity is necessary)**.** *There is an $\mathcal{M}$-sound sorted-core problem with an invalid pending goal and a* witnessed *but non-monotonic type τ such that erasing the τ-guards proves the goal.*

Proof. Take $\tau = \mathsf{unit}$ (single constructor tt) and as premise its own inversion axiom, emitted by the translation itself: $\forall x.\ \neg\mathsf{P}_{\mathsf{unit}}(x) \vee x \approx \hat{\mathsf{tt}}$. It is valid ($[\![\mathsf{unit}]\!] = \{[\![\mathsf{tt}]\!]\}$). The sort is witnessed ($\hat{\mathsf{tt}}$) but the variable x occurs naked in a positive equation, and indeed unit is genuinely non-monotonic: the axiom bounds the guard extension by one element. Erase the guard: $\forall x.\ x \approx \hat{\mathsf{tt}}$. Instantiating x by $\hat{\mathsf{O}}$ and by $\hat{\mathsf{S}}(\hat{\mathsf{O}})$ yields $\hat{\mathsf{O}} \approx \hat{\mathsf{S}}(\hat{\mathsf{O}})$: the invalid goal of Proposition 7.18 again follows. □

Remark 7.20 (The case-scrutinee counterexample, revisited). [CK18, §5.6] observes that the guards on the free variables of a case scrutinee can never be omitted, exhibiting a one-constructor type c with $\mathsf{I}^0(c : \mathsf{Set} := c_0 : c)$ where omitting the guard produces the axiom $\forall x.\ x = c_0 \wedge F\,x = c_0$ and thence an inconsistency. Through the lens of this section the diagnosis is exact: the conjunct $\forall x.\ x \approx c_0$ is the inversion content of a *singleton* — witnessed, maximally non-monotonic — type, i.e. precisely the situation of Proposition 7.19. The guards that *may* be dropped are those of Definition 7.16; the case-scrutinee guards at singleton and enumeration types are the paradigmatic non-examples, and Theorem 7.17 never licenses them. Conversely, the folklore practice of omitting guards at types like nat or $\mathsf{list}\,\mathsf{nat}$ is licensed, for the first time with a proof, by Lemma 7.15 together with Theorem 7.17.

Remark 7.21 (Scope, and comparison with the featherweight encodings). (i) Our erasure is *per-type*, like the lightweight encodings $\widetilde{\mathsf{g}}?$ of [CLS11, BBPS16]; the featherweight $\widetilde{\mathsf{g}}??$ additionally drops guards of non-naked *variables* at non-monotonic types. The analogous per-variable refinement is meaningful in our setting, but its correctness proof requires interpreting a kept guard as *false* on merged junk (the device of [BBPS16, Thm. 5.20]) inside our single-domain merge, where the same junk element is shared by all erased sorts; we leave this refinement to future work. (ii) The freedom to choose Σ_0 as *any* subset of the erasable sorts (cf. [BBPS16, Rem. 5.21]) is built into Theorem 7.17. (iii) The theorem is proved for the drop-policy monomorphic pipeline. With kept generic premises the problem is not a sorted core — binary T-atoms with variable type arguments remain — and erasure of the specialised part is not covered; the appropriate tool there would be a polymorphic monotonicity calculus in the style of [BBPS16, §6], which we also leave open. (iv) Erasure interacts with Remark 4.14: after erasure at an *erasable* type, the collapse phenomena that make guard omission unsound in the

implemented translation cannot occur, since erasable types admit models where the guard is total.

8 The composed pipeline

We assemble the stages and state the end-to-end guarantee.

Definition 8.1 (Pipeline)**.** Fix an admissible environment, premises P (valid closed propositions) and a closed goal φ_G. Fix parameters $K, \Delta_{\mathrm{tot}}, \Delta_c$ (defaults $3, 200, 16$). The *monomorphising pipeline* computes:

(P1) *Saturate:* a monomorphisation Θ by the bounded saturation of Definition 5.11, including instances of the implicit inductive premises (Definition 5.5);

(P2) *Instantiate:* $P' \coloneqq \mathrm{Mono}_\Theta(P)$, with policy *drop* (default for the stages below) or *keep*;

(P3) *Translate:* $(\Phi_0, \gamma) \coloneqq \mathrm{Transl}(P', \varphi_G)$ (Definition 4.6);

(P4) *Prune and specialise* (drop policy, first-order core only): $(\Phi_1, \gamma_1) \coloneqq \mathcal{S}(\Phi_0', \gamma)$ where Φ_0' prunes as in Lemma 6.3;

(P5) *Erase:* compute the erasable sorts (Definition 7.16: witnessed, and NV-safe or inductive-infinite — both syntactic checks), choose Σ_0 among them, and output $(\Phi_2, \gamma_2) \coloneqq (\mathcal{E}_{\Sigma_0}(\Phi_1), \mathcal{E}_{\Sigma_0}(\gamma_1))$.

The problem (Φ_2, γ_2) is handed to the ATPs. If a refutation is found, the premises used in it are mapped back through the instance bookkeeping $(\varphi_p \cdot \theta \mapsto p)$ and given to the certified reconstruction backend.

Theorem 8.2 (End-to-end soundness)**.** *Suppose the hypotheses of Lemmas 6.3 and 7.7 hold for the pipeline run (first-order core, witnessed quantified types) and Σ_0 consists of erasable sorts. If $\Phi_2 \vdash_{\mathrm{c}} \gamma_2$, then φ_G is valid in $\mathcal{M}$.*

Proof. By Theorem 5.6, (Φ_0, γ) is $\mathcal{M}$-sound; pruning removes axioms, preserving $\mathcal{M}$-soundness trivially; by Theorem 6.2, (Φ_1, γ_1) is $\mathcal{M}$-sound. By Lemma 7.7, $\Phi_1 \cup \{\mathrm{nnf}(\neg\gamma_1)\}$ is a sorted core, so by Theorem 7.17 it is Σ_0-inflatable, and Theorem 7.5 converts the assumed provability into validity of φ_G. □

Proposition 8.3 (Relative completeness of the back end)**.** *Stages (P4)–(P5) lose no proofs: if $\Phi_0' \vdash_{\mathrm{c}} \gamma$ then $\Phi_2 \vdash_{\mathrm{c}} \gamma_2$.*

Proof. Theorem 6.4 (totality by Lemma 6.3) and Proposition 7.3. □

Remark 8.4 (Where incompleteness is accepted). Stage (P2) with the drop policy is incomplete by design (Proposition 5.8), and provably cannot be repaired by a cleverer computable instance selection (Theorem 5.14); the keep policy restores completeness (Proposition 5.7) at the price of retaining the generic formulas, for which the erasure theory of Section 7 is not available (Remark 7.21(iii)). The pruning inside (P4) removes the application-typing axioms; for constant-headed terms their content is subsumed by the flattened typing axioms, so we expect no practical loss, but we make no formal claim. A hammer can, and should, hedge: CoqHammer already runs several ATPs in parallel with different translation options [CK18], and the natural deployment runs the drop-policy pipeline with erasure alongside a keep-policy or unmonomorphised run.

Remark 8.5 (Absolute safety). Independently of every theorem in this paper, the hammer architecture is fail-safe: the ATP output is used only to select premises, and the goal is re-proved from the selected premises by the certified search tactics and checked by the proof-assistant kernel [CK18]. What Theorem 8.2 adds is the guarantee that the optimised pipeline does not manufacture refutations of consistent problems — i.e. that ATP "successes" remain correlated with true goals, which is what makes the tool useful rather than merely safe. The necessity examples (Propositions 7.18 and 7.19) show this guarantee is not academic: unrestricted guard erasure produces exactly such phantom refutations.

Remark 8.6 (Implementation notes). In CoqHammer's architecture the pipeline slots in as follows. Stage (P1)–(P2) operates on the CIC_0 representation after premise selection and before the translation proper (in `coq_transl.ml` terms: between reinitialisation of the definition table and retranslation), with instances entered as new definitions with content-derived names so that per-name axiom caching continues to work; the instance-to-premise name map rides along for reconstruction. Stage (P4) generalises the existing `tconst`-specialisation and arity optimisation of `tptp_out.ml` [CK18, §5.5], with the bridging axioms removed as analysed in Section 6; stage (P5) is new, and its erasability check needs only the inductive declarations and the instance set. Suggested options: `Set Hammer Monomorphisation` (policy and bounds), `Set Hammer GuardSpecialization`, `Set Hammer GuardErasure`. Evaluation should follow the ablation grid baseline / +specialisation / +monomorphisation / +monomorphisation+specialisation / +all, on the standard `eval/` benchmark, measuring ATP-provability and reconstruction rates per prover.

9 Related work

Soundness of the CoqHammer translation. The implemented translation is described in [CK18], which states plainly that it "is neither sound nor complete" and confines the theoretical discussion to §5.6, including the guard counterexample revisited in Remark 7.20. The rigorous core is [Cza18]: a shallow embedding of proof-irrelevant Pure Type Systems into *minimal intuitionistic* first-order logic, proved sound by a constructive, proof-theoretic reconstruction argument over η-long normal derivations. Our Section 4 complements that result with a classical, model-theoretic soundness theorem for a CIC-style fragment with inductive types, in which the guard predicate is interpreted as set membership. The two approaches are genuinely different in kind: the reconstruction argument yields proof terms but, as [Cza18, §6] notes, is incompatible with the model-theoretic methods of monotonicity inference, whose adaptation to dependent type theory is posed there as an open problem. Section 7 answers that problem for the monomorphic fragment. The set-theoretic models we rely on are those of Werner, Aczel and Lee–Werner [Wer97, Acz99, LW11].

Type encodings and monotonicity. Type guards and tags for encoding sorted logics in untyped first-order logic go back a long way; systematic soundness analyses appear in Claessen, Lilliestrom and Smallbone [CLS11], who introduced monotonicity inference with the naked-variable calculus and its SAT-based refinement, and in Blanchette, Böhme, Popescu and Smallbone [BBPS16], who extended the framework to polymorphism, introduced the lightweight and featherweight encodings, and mechanised the monomorphic metatheory in Isabelle/HOL [BP13]. Couchot and Lescuyer [CL07] and Leino and Rümmer [LR10] studied guard- and tag-style encodings of polymorphism for SMT and intermediate verification languages. Our Section 7 transplants the monotonicity method to the guarded unsorted problems produced from dependent type theory; the technical novelties forced by the setting are the sorted-core extraction and relativisation (Theorem 7.9), the single-domain merge in Theorem 7.17, the explicit inhabitation side condition (many-sorted logic assumes nonempty sorts,

CIC does not — Proposition 7.18), and the internal infinity criterion via the translation's own injectivity and discrimination axioms (Lemma 7.15), which closes the introspection soundness gap noted in [BBPS16, §7.4].

Monomorphisation. Heuristic monomorphisation for Sledgehammer is defined in [BBPS16, §5.6, §7.1], with the iteration and size bounds we adopt; its incompleteness is remarked there without proof. Bobot and Paskevich [BP11] proved that complete monomorphisation is uncomputable for polymorphic first-order logic and designed the symbol-discrimination transformation Dis, whose keep-the-generic-formula design mirrors our keep policy. Theorem 5.14 adapts their combinatory-logic reduction to CIC^- with a semantic notion of consequence, adding the decidability analysis of the instance-set fragment (Horn saturation over reflected types) in full. Empirical comparisons across seventeen provers [DVBW22] found monomorphic encodings superior even to native polymorphic prover support (cf. the polymorphic extensions of Vampire [BR20]); this motivates investing in monomorphisation for CoqHammer rather than waiting for TF1-native back ends. For dependent type theory, Lean-auto [QCBA25] instantiates "constant instances" including dependent arguments during its preprocessing, which corresponds to our general instantiations (Remark 5.10); its published description argues correctness informally, while our Theorem 5.6 gives the semantic soundness statement and proof for the CIC fragment. Premise selection for hammers in dependent type theory [ZCA+26] is orthogonal: it chooses P, we transform it.

Arity and predicate optimisations. The flattening steps (S3)–(S4) are folklore for hammers and were analysed for higher-order-to-first-order translations by Meng and Paulson [MP08]; CoqHammer performs them with bridging axioms [CK18, §5.5]. The contribution of Section 6 is the uniform soundness principle covering them together with guard specialisation and type-argument fusion, and the faithfulness theorem for the bridge-free versions on the first-order core.

10 Conclusion

We gave a precise formulation, over a stratified fragment CIC^- of the Calculus of Inductive Constructions, of the three optimisations that separate the current CoqHammer translation from the state of the art of hammers for simple type theory: heuristic monomorphisation, bridge-free signature specialisation, and monotonicity-based guard erasure. All three were proved sound with respect to a set-theoretic standard model, via a self-contained truth lemma for the guarded translation in which the guard predicate is membership. The main technical results are: soundness of monomorphisation by semantic instantiation, independent of the instance-selection heuristic, with conservativity for the keep policy and the uncomputability of complete instance selection; faithfulness of total specialisation on the first-order core without bridging axioms; and the guard-erasure theorem — guards of witnessed, monotonic types may be erased, with monotonicity certified either by naked-variable analysis or by an internal infinity criterion that the translation's own injectivity and discrimination axioms establish for recursive inductive types. Both side conditions were shown necessary, and the known counterexample to guard omission in the CoqHammer literature was identified as a non-monotonicity phenomenon.

Several directions remain open. *Theory:* the per-variable (featherweight) refinement of erasure in the single-domain setting; a polymorphic monotonicity calculus applicable to kept generic premises; faithfulness for shape-specialised guards over open types; extending CIC^- with indexed inductive families, cumulativity and fixpoints (we expect the semantic method to scale, at the cost of a heavier model section); and a mechanisation, for which the Isabelle

formalisation of the monomorphic sorted metatheory [BP13] is a natural starting point. *Practice:* implementing the pipeline in CoqHammer behind options, with the ablation evaluation of Remark 8.6; the literature [BBPS16, DVBW22] predicts the largest gains from the combination of drop-policy monomorphisation with erasure, which is precisely the configuration our theorems license.

References

[Acz99] Peter Aczel. On relating type theories and set theories. In *Types for Proofs and Programs (TYPES 1998)*, volume 1657 of *LNCS*, pages 1–18. Springer, 1999.

[Bar84] Henk Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, revised edition, 1984.

[Bar92] Henk Barendregt. Lambda calculi with types. In *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.

[BBPS16] Jasmin Christian Blanchette, Sascha Böhme, Andrei Popescu, and Nicholas Smallbone. Encoding monomorphic and polymorphic types. *Logical Methods in Computer Science*, 12(4:13):1–52, 2016.

[BP11] François Bobot and Andrei Paskevich. Expressing polymorphic types in a many-sorted language. In *Frontiers of Combining Systems (FroCoS 2011)*, volume 6989 of *LNCS*, pages 87–102. Springer, 2011. Extended version: INRIA research report, https://inria.hal.science/inria-00591414/.

[BP13] Jasmin Christian Blanchette and Andrei Popescu. Mechanizing the metatheory of Sledgehammer. In *Frontiers of Combining Systems (FroCoS 2013)*, volume 8152 of *LNCS*, pages 245–260. Springer, 2013.

[BR20] Ahmed Bhayat and Giles Reger. A polymorphic Vampire (short paper). In *Automated Reasoning (IJCAR 2020)*, volume 12167 of *LNCS*, pages 361–368. Springer, 2020.

[CK90] Chen Chung Chang and H. Jerome Keisler. *Model Theory*, volume 73 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, 3rd edition, 1990.

[CK18] Łukasz Czajka and Cezary Kaliszyk. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61(1–4):423–453, 2018.

[CL07] Jean-François Couchot and Stéphane Lescuyer. Handling polymorphism in automated deduction. In *Automated Deduction – CADE-21*, volume 4603 of *LNCS*, pages 263–278. Springer, 2007.

[CLS11] Koen Claessen, Ann Lilliestöm, and Nicholas Smallbone. Sort it out with monotonicity: Translating between many-sorted and unsorted first-order logic. In *Automated Deduction – CADE-23*, volume 6803 of *LNCS*, pages 207–221. Springer, 2011.

[Cza18] Łukasz Czajka. A shallow embedding of pure type systems into first-order logic. In *22nd International Conference on Types for Proofs and Programs (TYPES 2016)*, volume 97 of *LIPIcs*, pages 9:1–9:39. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.

[DVBW22] Martin Desharnais, Petar Vukmirović, Jasmin Blanchette, and Makarius Wenzel. Seventeen provers under the hammer. In *13th International Conference on Interactive Theorem Proving (ITP 2022)*, volume 237 of *LIPIcs*, pages 8:1–8:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022.

[Geu93] Herman Geuvers. *Logics and Type Systems*. PhD thesis, University of Nijmegen, 1993.

[LR10] K. Rustan M. Leino and Philipp Rümmer. A polymorphic intermediate verification language: Design and logical encoding. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2010)*, volume 6015 of *LNCS*, pages 312–327. Springer, 2010.

[LW11] Gyesik Lee and Benjamin Werner. Proof-irrelevant model of CC with predicative induction and judgmental equality. *Logical Methods in Computer Science*, 7(4:5):1–25, 2011.

[MP08] Jia Meng and Lawrence C. Paulson. Translating higher-order clauses to first-order clauses. *Journal of Automated Reasoning*, 40(1):35–60, 2008.

[PM93] Christine Paulin-Mohring. Inductive definitions in the system Coq: Rules and properties. In *Typed Lambda Calculi and Applications (TLCA 1993)*, volume 664 of *LNCS*, pages 328–345. Springer, 1993.

[QCBA25] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. Lean-auto: An interface between Lean 4 and automated theorem provers. In *Computer Aided Verification (CAV 2025)*, LNCS. Springer, 2025. arXiv:2505.14929.

[Wer94] Benjamin Werner. *Une théorie des constructions inductives*. PhD thesis, Université Paris 7, 1994.

[Wer97] Benjamin Werner. Sets in types, types in sets. In *Theoretical Aspects of Computer Software (TACS 1997)*, volume 1281 of *LNCS*, pages 530–546. Springer, 1997.

[ZCA+26] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a Lean hammer. In *International Conference on Learning Representations (ICLR 2026)*, 2026. arXiv:2506.07477.